

THE HAVEN PROTOCOL

DOCUMENT 4

ZERO-TRUST TECHNICAL GOVERNANCE & ABAC COMPLIANCE FRAMEWORK

Attribute-Based Authorization

Immutable Audit Infrastructure

Regulatory Policy Enforcement

Clinical Resilience & Emergency Governance

Document Classification: Technical Architecture / Security Governance / Compliance Infrastructure

Protocol: Haven Protocol

Document: 04 of the Haven Technical Governance Series

Architecture Domain: Zero-Trust Authorization, Policy Enforcement, Auditability & Regulatory Compliance

Primary Execution Model: In-Memory Authorization Sidecars / OPA-WASM Compatible Policy Runtime

Audit Model: Asynchronous Event Streaming + Cryptographic Merkle Batching + Periodic Ledger Anchoring

Operational States: NORMAL / DEGRADED / EMERGENCY BREAK-GLASS

Regulatory Abstraction Model: Declarative Statutory Policy Packs

Authorization Performance Objective: <10 ms authorization overhead

Core Principle:

No trust is inherited. Every consequential system action is evaluated against explicit attributes, policy state, operational context, and regulatory constraints before execution.

CONFIDENTIAL ARCHITECTURAL MATERIAL

This document defines the governance and compliance enforcement layer of the Haven Protocol.

It establishes the technical mechanisms through which authorization, regulatory policy, clinical safety, auditability, and emergency intervention are transformed from organizational procedures into continuously executable infrastructure controls.

The framework is designed for fragmented healthcare environments in which multiple hospitals, laboratories, pharmacies, insurers, health platforms, government entities, and clinical applications may participate in the same coordinated workflow without sharing a single centralized application or database.

Haven therefore does not rely upon institutional trust as an authorization primitive.

It relies upon **verifiable attributes, explicit policy evaluation, cryptographically observable events, controlled state transitions, and independently enforceable governance boundaries.**

ARCHITECTURAL DIRECTIVE

The Haven authorization layer SHALL operate as an enforcement plane positioned adjacent to participating application workloads.

Authorization decisions SHALL be compiled into an in-memory policy execution environment, including an OPA/WASM-compatible sidecar model where appropriate.

The synchronous request path SHALL NOT depend upon an external Policy Information Point through a network-bound HTTP or gRPC roundtrip.

The target authorization overhead SHALL remain **below 10 milliseconds** under defined production operating conditions.

Regulatory controls SHALL be represented as versioned, declarative policy artifacts rather than permanently embedded statutory interpretations inside application binaries.

Audit generation SHALL remain decoupled from synchronous transaction execution.

Clinical workflows SHALL operate through an explicit operational state machine:

NORMAL → DEGRADED → EMERGENCY BREAK-GLASS

No critical patient-care workflow shall be rendered permanently unavailable solely because a policy dependency, identity attribute source, or authorization infrastructure component becomes temporarily unreachable.

Every emergency exception SHALL instead create an auditable, attributable, cryptographically observable governance event.

GOVERNANCE OBJECTIVE

Haven converts compliance from a periodic review activity into a continuously executable infrastructure property.

The resulting architecture is intended to ensure that:

Authorization is evaluated before execution.

Regulatory policy is evaluated as machine-readable governance.

Exceptions are explicitly attributable.

Audit records cannot be silently rewritten.

Clinical continuity survives controlled infrastructure degradation.

Emergency authority is bounded rather than unlimited.

Policy changes are versioned and independently observable.

Every consequential state transition leaves an evidentiary trail.

1. ZERO-TRUST GOVERNANCE ARCHITECTURE

1.1 Purpose

The Haven Zero-Trust Technical Governance & ABAC Compliance Framework defines the security control plane responsible for determining whether a proposed system action may occur, under which conditions it may occur, what regulatory constraints apply, and what evidence must subsequently exist to prove that the decision was legitimate.

The framework treats authorization as an **architectural execution function**, not an application-level convenience.

A request entering the Haven coordination environment is therefore not considered authorized merely because:

- the requesting application is registered;
- the user has previously authenticated;
- the institution is trusted;
- the request originates from an approved network;
- the same user successfully performed an earlier action; or
- a downstream application has historically permitted the operation.

Authorization is evaluated against the attributes relevant to the **specific transaction, actor, resource, purpose, environment, jurisdiction, workflow state, and regulatory policy version**.

1.2 Core Authorization Model

The Haven authorization decision SHALL conceptually evaluate:

```
AUTHORIZATION DECISION
=
SUBJECT ATTRIBUTES
+
RESOURCE ATTRIBUTES
+
ACTION ATTRIBUTES
+
ENVIRONMENT ATTRIBUTES
+
PURPOSE ATTRIBUTES
+
WORKFLOW STATE
+
JURISDICTION
+
REGULATORY POLICY
+
RISK CONDITIONS
```

+
TEMPORAL CONDITIONS

The resulting decision SHALL resolve into an enforceable policy outcome:

ALLOW
DENY
ALLOW_WITH_CONSTRAINTS
REQUIRE_STEP_UP
REQUIRE_BREAK_GLASS
DEFER

The policy engine SHALL NOT require a synchronous call to a remote Policy Information Point during the core authorization path.

Instead, attributes required for real-time decisions SHALL be supplied through locally available, cryptographically protected context and controlled attribute caches.

1.3 ABAC Execution Plane

The authorization architecture SHALL consist of five logical components:

A. Policy Administration Point — PAP

Responsible for authoring, versioning, approving, signing, and distributing policy definitions.

Policies may include:

- organizational authorization policies;
- clinical workflow policies;
- data-access policies;
- jurisdictional restrictions;
- consent requirements;
- purpose-of-use constraints;
- emergency-access policies;
- retention controls;
- regulatory policy mappings.

B. Policy Decision Point — PDP

The PDP performs the actual authorization evaluation.

The PDP SHALL execute inside an in-memory policy runtime and SHALL be capable of operating independently of synchronous external network calls.

A WASM-compiled policy package or equivalent embedded policy runtime may be used to achieve deterministic execution and low authorization latency.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Performance requirement:

Target authorization overhead: <10 ms
External PIP dependency in request path: PROHIBITED
Network-bound policy lookup in request path: PROHIBITED

C. Policy Enforcement Point — PEP

The PEP exists at the application, API gateway, service mesh, workflow, or protocol boundary where the authorization decision becomes enforceable.

The PEP SHALL:

1. construct the authorization request context;
2. invoke the local PDP;
3. receive the policy decision;
4. enforce the resulting decision;
5. emit the corresponding audit event asynchronously.

The PEP SHALL NOT independently reinterpret statutory policy.

D. Policy Information Point — PIP

The PIP supplies attributes required for policy evaluation.

Examples include:

```
subject.role  
subject.organization  
subject.license_status  
subject.identity_assurance  
resource.classification  
resource.owner  
resource.jurisdiction  
request.purpose  
request.workflow  
request.timestamp  
environment.risk_level  
consent.status  
emergency.status
```

PIP data may originate from external systems, but those systems SHALL NOT become mandatory synchronous dependencies for every authorization decision.

E. Policy Enforcement Evidence Stream

Every consequential authorization decision SHALL generate an immutable audit event containing sufficient information to reconstruct:

WHO
WHAT
WHEN
WHERE
WHY
UNDER WHICH POLICY
WITH WHICH ATTRIBUTES
UNDER WHICH SYSTEM STATE
WITH WHAT RESULT

The audit event SHALL be emitted asynchronously and SHALL not block the clinical or transactional request path.

1.4 Regulatory Policy Abstraction

Statutory frameworks such as the **Nigeria Data Protection Act 2023 (NDPA)** and applicable international frameworks SHALL be represented as configurable policy packs.

The architecture SHALL NOT embed statutory legal provisions directly into compiled application execution gates.

Instead:

```
STATUTORY REQUIREMENT
    ↓
LEGAL/POLICY INTERPRETATION
    ↓
DECLARATIVE POLICY RULE
    ↓
SIGNED POLICY PACKAGE
    ↓
WASM / IN-MEMORY RUNTIME
    ↓
PEP ENFORCEMENT
    ↓
AUDIT EVIDENCE
```

This separation permits regulatory policies to evolve without requiring the underlying Haven application binaries to be rewritten.

A regulatory policy package SHALL possess an independent version identifier, effective date, jurisdiction, authority classification, approval state, and cryptographic integrity identifier.

This creates a **statutory abstraction layer** between changing law and stable protocol infrastructure.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

1.5 Architectural Compliance Moat

The compliance moat is created not by claiming permanent legal compliance, but by making governance infrastructure **continuously executable, versioned, observable, and difficult to bypass without generating evidence.**

A participating system therefore inherits a governance execution model rather than merely receiving a compliance checklist.

The competitive advantage compounds through:

- policy version history;
- jurisdiction-specific rule libraries;
- machine-readable regulatory mappings;
- authorization telemetry;
- immutable audit evidence;
- emergency-access histories;
- enforcement patterns;
- integration-specific policy configurations;
- cryptographic provenance of policy decisions.

The architecture consequently transforms regulatory complexity into an infrastructure layer that becomes increasingly difficult for a competing system to reproduce once deeply integrated across participating healthcare networks.

2. ABAC EXECUTION LOGIC ARRAY

2.1 Authorization Evaluation Contract

Haven SHALL treat every authorization request as a structured policy evaluation rather than a binary identity check.

The minimum authorization context SHALL contain the following logical dimensions:

SUBJECT

- subject_id
- role
- organization
- professional_status
- assurance_level
- delegation_state

RESOURCE

- resource_id
- resource_type
- classification
- jurisdiction
- ownership_domain
- sensitivity_level

ACTION

- action_id
- operation
- requested_scope
- transaction_type

PURPOSE

- purpose_code
- declared_purpose
- treatment_context
- secondary_use_indicator

ENVIRONMENT

- timestamp
- device_trust
- network_context
- risk_score
- operational_state

GOVERNANCE

- policy_version
- jurisdiction
- consent_state
- regulatory_state
- emergency_state

The PDP SHALL evaluate the complete context against the currently active policy bundle.

Missing attributes SHALL NOT automatically produce an unrestricted authorization.

Where an attribute is mandatory for the requested operation, the policy SHALL explicitly determine whether its absence results in:

DENY
DEFER
STEP_UP
BREAK_GLASS

The outcome SHALL therefore be policy-defined rather than implementation-defined.

2.2 Execution Logic Array

The canonical authorization evaluation SHALL follow a deterministic sequence:

```
[01] INGEST REQUEST CONTEXT
      ↓
[02] NORMALIZE ATTRIBUTES
      ↓
[03] VALIDATE ATTRIBUTE INTEGRITY
      ↓
[04] RESOLVE SUBJECT ATTRIBUTES
      ↓
[05] RESOLVE RESOURCE ATTRIBUTES
      ↓
[06] RESOLVE ACTION + PURPOSE
      ↓
[07] LOAD ENVIRONMENT STATE
      ↓
[08] IDENTIFY JURISDICTION
      ↓
[09] SELECT POLICY BUNDLE
      ↓
[10] EVALUATE REGULATORY CONSTRAINTS
      ↓
[11] EVALUATE ORGANIZATIONAL RULES
      ↓
[12] EVALUATE CLINICAL WORKFLOW RULES
      ↓
[13] EVALUATE RISK CONDITIONS
      ↓
[14] EVALUATE EMERGENCY CONDITIONS
      ↓
[15] RESOLVE POLICY PRIORITY
      ↓
[16] ISSUE DECISION
      ↓
[17] ENFORCE DECISION
      ↓
[18] EMIT ASYNCHRONOUS AUDIT EVENT
```

Steps 01–17 SHALL remain within the synchronous authorization path.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Step 18 SHALL be decoupled from the request execution path through the audit event queue.

2.3 Policy Precedence

Haven SHALL implement explicit policy precedence to prevent lower-priority permissions from overriding higher-order restrictions.

The conceptual precedence hierarchy SHALL be:

```
L0  SYSTEM SAFETY
L1  EMERGENCY CLINICAL SAFETY
L2  STATUTORY / REGULATORY CONSTRAINT
L3  DATA SUBJECT / CONSENT CONSTRAINT
L4  ORGANIZATIONAL GOVERNANCE
L5  CLINICAL WORKFLOW POLICY
L6  RESOURCE POLICY
L7  SUBJECT ROLE PERMISSION
L8  DEFAULT APPLICATION PERMISSION
```

A lower layer SHALL NOT override a higher layer unless the higher layer explicitly defines an exception pathway.

This prevents an application-level permission from silently defeating a statutory restriction or a clinical safety control.

2.4 Attribute Integrity

ABAC decisions are only as trustworthy as the attributes upon which they depend.

Haven SHALL therefore distinguish between:

```
TRUSTED
VERIFIED
CACHED
STALE
UNKNOWN
REVOKED
CONFLICTING
```

Each attribute SHALL carry metadata describing its provenance and freshness.

A representative attribute object SHALL conform conceptually to:

```
{
  "name": "professional_status",
  "value": "active",
  "source": "licensed_registry",
  "assurance": "verified",
  "issued_at": "2026-09-03T08:00:00Z",
  "expires_at": "2026-09-03T20:00:00Z",
```

```
"policy_version": "NDPA-NG-2026.09",  
"integrity": "verified"  
}
```

The PDP SHALL be capable of applying different authorization behavior based upon attribute assurance.

For example, a verified professional credential may authorize a normal workflow while an expired or unknown credential may require step-up verification or a controlled emergency pathway.

2.5 In-Memory Policy Compilation

The authorization runtime SHALL compile active policy packages into an in-memory executable representation.

The production architecture SHOULD support OPA/WASM or an equivalent deterministic policy execution mechanism.

The critical design requirement is not the vendor implementation itself, but the execution property:

```
NO REMOTE POLICY DEPENDENCY  
NO SYNCHRONOUS PIP NETWORK ROUNDTRIP  
NO DATABASE QUERY FOR EVERY DECISION  
NO EXTERNAL AUTHORIZATION SERVICE IN CORE PATH
```

Policy bundles SHALL be loaded, validated, cryptographically verified, compiled, and activated independently of individual clinical requests.

A policy update SHALL therefore follow:

```
POLICY AUTHORIZING  
↓  
VALIDATION  
↓  
APPROVAL  
↓  
SIGNING  
↓  
DISTRIBUTION  
↓  
LOCAL VERIFICATION  
↓  
COMPILATION  
↓  
ATOMIC ACTIVATION
```

A malformed or invalid policy package SHALL never partially replace an active policy package.

2.6 Authorization Performance Envelope

The authorization subsystem SHALL be engineered against a target overhead of **less than 10 milliseconds**.

This threshold applies to the authorization computation and local policy evaluation overhead rather than downstream clinical service latency.

Performance instrumentation SHALL measure at minimum:

```
policy_load_time
attribute_resolution_time
policy_evaluation_time
decision_serialization_time
PEP_enforcement_time
audit_enqueue_time
```

The audit enqueue operation SHALL be non-blocking wherever the workflow permits.

A slow or temporarily unavailable ledger anchoring mechanism SHALL not automatically increase the latency of the clinical transaction.

3. POLICY ENFORCEMENT BOUNDARY

3.1 Enforcement Point Placement

The Policy Enforcement Point SHALL be positioned at every boundary where an unauthorized state transition could materially affect protected healthcare information, clinical workflow, financial settlement, identity state, or regulatory obligations.

Candidate enforcement boundaries include:

```
API GATEWAYS
SERVICE MESHES
WORKFLOW ORCHESTRATORS
CLINICAL APPLICATION ADAPTERS
DATA ACCESS LAYERS
INTEROPERABILITY CONNECTORS
PAYMENT/SETTLEMENT ADAPTERS
ADMINISTRATIVE CONTROL PLANES
```

The PEP SHALL be incapable of granting itself authority.

Its responsibility is enforcement, not policy interpretation.

3.2 Decision Object

The PDP SHALL return a structured decision object.

A representative decision SHALL contain:

```
{
  "decision": "ALLOW",
  "decision_id": "dec_01J...",
  "policy_version": "haven-policy-2026.09.03",
  "policy_hash": "sha256:...",
  "jurisdiction": "NG",
  "constraints": [],
  "obligations": [],
  "expires_at": "2026-09-03T09:20:00Z"
}
```

For constrained authorization:

```
{
  "decision": "ALLOW_WITH_CONSTRAINTS",
  "constraints": [
    "READ_ONLY",
    "PURPOSE_LIMITED",
    "NO_SECONDARY_USE"
  ]
}
```

The consuming application SHALL enforce all returned obligations.

An application SHALL NOT silently discard a constraint because the underlying business operation remains technically possible.

3.3 Explicit Denial Semantics

A denial SHALL be machine-readable and attributable.

The system SHALL distinguish among:

```
POLICY_DENIED
ATTRIBUTE_MISSING
ATTRIBUTE_INVALID
CONSENT_REQUIRED
PURPOSE_NOT_PERMITTED
JURISDICTION_RESTRICTED
CREDENTIAL_REVOKED
POLICY_VERSION_INVALID
SYSTEM_STATE_RESTRICTED
EMERGENCY_AUTHORIZATION_REQUIRED
```

These distinctions are necessary because an ordinary denial and a clinical emergency are fundamentally different governance conditions.

The error response SHALL therefore communicate enough information for the requesting system to determine the correct next action without exposing sensitive policy internals.

3.4 Mermaid Authorization Sequence

```
sequenceDiagram
    participant C as Client
    participant PEP as Policy Enforcement Point
    participant PDP as In-Memory PDP
    participant CACHE as Attribute Cache
    participant POLICY as Policy Bundle
    participant Q as Audit Queue

    C->>PEP: Protected Request
    PEP->>CACHE: Read Local Attributes
    CACHE-->>PEP: Attribute Context
    PEP->>PDP: Authorization Context
    PDP->>POLICY: Evaluate Active Rules
    POLICY-->>PDP: Policy Result
    PDP-->>PEP: Decision + Constraints
    PEP->>Q: Async Audit Event
    PEP-->>C: Authorized Response
```

The diagram intentionally contains no synchronous external PIP request.

External attribute sources may update the local attribute state asynchronously, while the core authorization path remains operationally independent.

4. THREE-TIER OPERATIONAL STATE ENGINE

4.1 State Model

Haven SHALL implement an explicit operational state engine consisting of exactly three primary authorization states:

```
NORMAL
↓
DEGRADED
↓
EMERGENCY BREAK-GLASS
```

The system SHALL NOT implicitly transition between these states based upon undocumented application behavior.

Every transition SHALL have a defined trigger, authorized transition mechanism, policy consequence, and audit representation.

4.2 NORMAL STATE

NORMAL represents the standard operating condition.

In this state:

- current policy bundles are available;
- required attributes are available at acceptable assurance;
- policy evaluation is functioning;
- authorization decisions are evaluated normally;
- audit events are successfully queued;
- regulatory controls operate under their ordinary policy definitions.

The expected execution model is:

```
REQUEST
↓
ATTRIBUTE CONTEXT
↓
LOCAL PDP
↓
POLICY EVALUATION
↓
ALLOW / DENY / CONSTRAIN
↓
ASYNC AUDIT
```

Normal operation SHALL impose no unnecessary clinical friction.

4.3 DEGRADED STATE

DEGRADED SHALL activate when one or more dependencies required for optimal authorization confidence become temporarily unavailable without constituting a clinical emergency.

Examples include:

- temporary registry unavailability;
- delayed attribute synchronization;
- intermittent connectivity;
- unavailable remote identity provider;
- delayed policy distribution;
- temporary audit infrastructure impairment.

The system SHALL use encrypted attribute caching under explicitly configurable TTL values.

A cached attribute SHALL NEVER be treated as equivalent to a currently verified attribute merely because it remains cryptographically intact.

Each cached attribute SHALL therefore contain:

ATTRIBUTE VALUE
SOURCE
ISSUANCE TIME
EXPIRATION TIME
CACHE TTL
ASSURANCE LEVEL
SIGNATURE / INTEGRITY STATE

Policy SHALL determine what actions remain permissible using cached attributes.

For example:

FRESH VERIFIED ATTRIBUTE
→ NORMAL AUTHORIZATION

VALID CACHED ATTRIBUTE
→ LIMITED AUTHORIZATION

STALE ATTRIBUTE
→ RESTRICTED / STEP-UP

MISSING CRITICAL ATTRIBUTE
→ BREAK-GLASS EVALUATION WHERE CLINICALLY JUSTIFIED

4.4 Emergency Break-Glass

EMERGENCY BREAK-GLASS exists to preserve critical patient-care continuity when ordinary authorization dependencies cannot safely resolve a clinically necessary action.

Break-glass SHALL NOT mean unrestricted access.

It SHALL mean **temporarily expanded authority under heightened accountability**.

A valid break-glass event SHALL require, where technically feasible:

AUTHORIZED ACTOR
+
DECLARED EMERGENCY PURPOSE
+
PATIENT / WORKFLOW CONTEXT
+
JUSTIFICATION
+
TIME-BOUND AUTHORITY
+
ENHANCED AUDIT
+
POST-EVENT REVIEW

The resulting authority SHALL automatically expire after its configured validity period unless renewed through an explicit policy pathway.

4.5 No Uncontrolled Fail-Closed

Haven SHALL NOT permit infrastructure degradation to create an undocumented hard FAIL_CLOSED condition that could obstruct a critical patient-care workflow without an audited emergency pathway.

The governing principle is:

SAFETY RESTRICTION ≠ UNCONTROLLED CLINICAL BLOCK

If normal authorization cannot be completed and the workflow is classified as clinically critical, the system SHALL evaluate the Emergency Break-Glass policy.

A break-glass decision SHALL be more restrictive in accountability, not less restrictive in governance.

Every emergency authorization SHALL produce an auditable event identifying:

WHO ACTIVATED IT
WHY IT WAS ACTIVATED
WHAT WAS ACCESSED
WHICH POLICY PERMITTED IT
WHEN AUTHORITY BEGAN
WHEN AUTHORITY EXPIRED
WHAT ACTIONS OCCURRED
WHETHER POST-EVENT REVIEW IS REQUIRED

4.6 State Transition Matrix

Current State	Trigger	Result	Authorization Behavior
NORMAL	Healthy dependencies	NORMAL	Standard ABAC
NORMAL	Attribute/PIP degradation	DEGRADED	Cached attributes + TTL
DEGRADED	Dependency recovery	NORMAL	Fresh attribute validation
DEGRADED	Critical clinical requirement	BREAK-GLASS evaluation	Emergency policy
BREAK-GLASS	Authority expires	DEGRADED/NORMAL	Re-evaluation required
BREAK-GLASS	Emergency ends	NORMAL/DEGRADED	Authority revoked

The exact transition SHALL be determined by policy and system health telemetry.

No component SHALL independently declare itself permanently exempt from the state engine.

5. ENCRYPTED ATTRIBUTE CACHING & DEGRADATION CONTROL

5.1 Purpose

The degraded operating model exists to preserve controlled system functionality when authoritative attribute providers, identity services, registries, interoperability endpoints, or connectivity infrastructure are temporarily unavailable.

Degraded operation SHALL NOT become a secondary authorization architecture.

It SHALL remain governed by the same ABAC policy model used during normal operation, with the principal difference being the **assurance and freshness of the attributes available to the PDP**.

The central design principle is:

```
ATTRIBUTE UNAVAILABILITY
    ↓
DO NOT BYPASS POLICY
    ↓
USE CONTROLLED CACHED STATE
    ↓
APPLY TTL + ASSURANCE CONSTRAINTS
    ↓
RESTRICT AUTHORITY WHERE REQUIRED
    ↓
ESCALATE CLINICAL CRITICALITY
    ↓
BREAK-GLASS ONLY WHEN POLICY PERMITS
```

5.2 Attribute Cache Architecture

The local attribute cache SHALL be encrypted at rest and protected from unauthorized modification.

A logical cache record SHALL contain:

```
{
  "attribute_id": "attr_01J...",
  "subject_id": "sub_01J...",
  "attribute": "professional_status",
  "value": "active",
  "source": "authorized_registry",
  "assurance": "verified",
  "issued_at": "2026-09-03T08:00:00Z",
  "fresh_until": "2026-09-03T12:00:00Z",
  "cache_expires_at": "2026-09-03T14:00:00Z",
  "policy_scope": [
    "clinical_read",
```

```

        "clinical_write"
    ],
    "integrity": {
        "algorithm": "SHA-256",
        "status": "verified"
    }
}

```

The cache SHALL distinguish between **freshness** and **integrity**.

A cryptographically intact attribute may still be too old to authorize a sensitive operation.

Therefore:

INTEGRITY = "Is this attribute authentic and unmodified?"

FRESHNESS = "Is this attribute sufficiently current for this operation?"

Both dimensions SHALL be evaluated independently.

5.3 Configurable TTL Governance

TTL values SHALL be policy-controlled rather than universally fixed.

A low-risk attribute may tolerate a longer cache period than a high-risk attribute.

Illustrative policy:

LOW-RISK ATTRIBUTE
TTL → longer permitted window

CLINICAL CREDENTIAL
TTL → short permitted window

CONSENT STATE
TTL → highly restricted freshness window

EMERGENCY AUTHORITY
TTL → extremely short / event-bound

REGULATORY STATUS
TTL → policy-defined by jurisdiction

The policy engine SHALL evaluate:

```

attribute_age
required_freshness
resource_sensitivity
action_sensitivity
clinical_criticality
jurisdiction

```

operational_state

before accepting cached data.

5.4 Cache Poisoning Protection

The attribute cache SHALL be treated as a security-sensitive component.

The following controls SHALL apply:

SIGNED ATTRIBUTE RECORDS
ENCRYPTED STORAGE
INTEGRITY VERIFICATION
SOURCE PROVENANCE
VERSION IDENTIFICATION
EXPIRATION ENFORCEMENT
REPLAY PROTECTION
REVOCATION SUPPORT
ACCESS CONTROL

A locally cached attribute SHALL never become authoritative merely because it exists locally.

Where a revocation signal is received, the cached attribute SHALL be invalidated or downgraded according to policy.

5.5 Stale Attribute Behavior

The system SHALL classify cached attributes according to age:

FRESH
VALID-CACHED
STALE
EXPIRED
REVOKED
UNKNOWN

Illustrative decision logic:

```
IF attribute = FRESH
    → evaluate normally

IF attribute = VALID-CACHED
    → evaluate degraded policy

IF attribute = STALE
    → restrict sensitive operations

IF attribute = EXPIRED
    → deny or escalate according to workflow policy

IF attribute = REVOKED
```

→ immediate invalidation

IF attribute = UNKNOWN

→ initiate appropriate recovery or break-glass evaluation

The PDP SHALL never silently convert an expired attribute into a valid one.

6. EMERGENCY BREAK-GLASS GOVERNANCE

6.1 Principle

Emergency authorization is a controlled exception mechanism for situations in which ordinary authorization cannot be completed without creating unacceptable clinical risk.

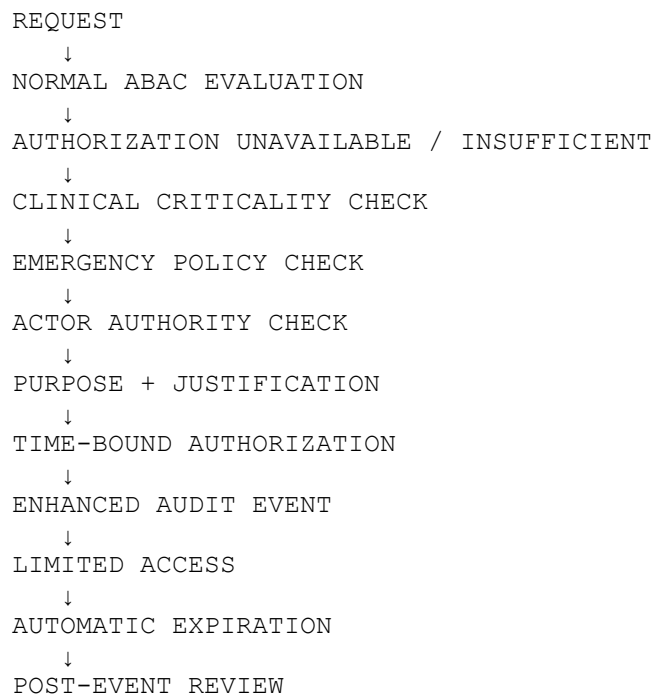
Break-glass SHALL therefore be designed as a **higher-observability authorization mode**, not as a security bypass.

The architecture SHALL assume that emergency access will eventually be used and shall make its use:

- explicit;
- attributable;
- time-limited;
- policy-constrained;
- cryptographically auditable;
- reviewable after the event.

6.2 Break-Glass Activation Logic

The conceptual execution sequence SHALL be:



A break-glass mechanism SHALL not activate solely because an application developer wants to avoid a denial.

The triggering condition must satisfy the applicable emergency policy.

6.3 Break-Glass JSON Contract

A production implementation SHALL expose a machine-readable emergency authorization object.

```
{
  "request_id": "req_01J...",
  "subject_id": "clinician_01J...",
  "resource_id": "patient_01J...",
  "action": "READ_CLINICAL_RECORD",
  "purpose": "EMERGENCY_TREATMENT",
  "emergency": true,
  "justification": "Immediate clinical intervention required.",
  "requested_scope": [
    "CURRENT_MEDICATIONS",
    "ALLERGIES",
    "RELEVANT_CLINICAL_HISTORY"
  ],
  "requested_duration_seconds": 900,
  "timestamp": "2026-09-03T08:30:00Z"
}
```

The corresponding decision SHALL explicitly identify the emergency authority:

```
{
  "decision": "REQUIRE_BREAK_GLASS",
  "break_glass": {
    "status": "APPROVED",
    "authority_id": "bg_01J...",
    "scope": [
      "CURRENT_MEDICATIONS",
      "ALLERGIES",
      "RELEVANT_CLINICAL_HISTORY"
    ],
    "issued_at": "2026-09-03T08:30:01Z",
    "expires_at": "2026-09-03T08:45:01Z",
    "review_required": true
  }
}
```

The authorization SHALL expire automatically.

6.4 Break-Glass Error Schema

Where emergency authorization is required but has not yet been activated:

```
{
  "error": {
    "code": "EMERGENCY_AUTHORIZATION_REQUIRED",
    "message": "This workflow requires emergency authorization.",
    "request_id": "req_01J...",
    "policy_version": "haven-policy-2026.09.03",
    "state": "DEGRADED",
    "break_glass_available": true,
    "next_action": "ACTIVATE_BREAK_GLASS"
  }
}
```

Where break-glass has expired:

```
{
  "error": {
    "code": "BREAK_GLASS_EXPIRED",
    "message": "Emergency authorization has expired.",
    "request_id": "req_01J...",
    "authority_id": "bg_01J...",
    "next_action": "REAUTHORIZE"
  }
}
```

These error responses SHALL remain intentionally limited.

They SHALL provide enough information for the authorized workflow to proceed without disclosing sensitive internal policy logic.

6.5 Break-Glass Audit Requirements

Every activation SHALL generate a high-priority audit event.

The minimum event SHALL capture:

```
event_id
authority_id
request_id
subject_id
actor_id
resource_id
purpose
justification
policy_version
jurisdiction
system_state
activation_timestamp
expiration_timestamp
scope
decision
attribute_assurance
```

The event SHALL be queued asynchronously for cryptographic audit processing.

Break-glass SHALL never depend upon synchronous ledger anchoring before clinical execution.

This separation preserves the critical distinction between:

permission to act now

and

cryptographic proof of what happened afterward.

6.6 Post-Event Review

Break-glass activity SHALL automatically generate a review obligation where required by policy.

The review engine MAY evaluate:

Was the emergency declaration justified?
Was the requested scope proportionate?
Was access limited to the emergency?
Did access continue after the emergency ended?
Were additional resources accessed?
Was the actor appropriately authorized?
Was the event correctly recorded?

Repeated use by the same actor, institution, workflow, or integration SHOULD be available as an aggregated governance signal.

This permits Haven to identify systemic dependency failures rather than treating each emergency event as an isolated exception.

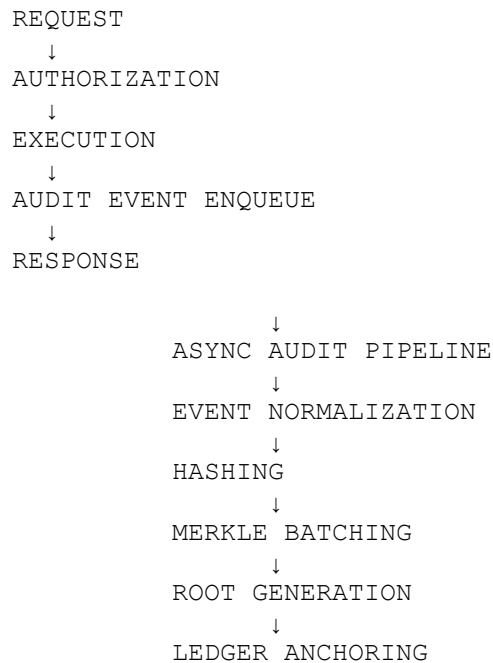
7. DECOUPLED ASYNCHRONOUS AUDIT ARCHITECTURE

7.1 Audit Design Principle

The cryptographic audit mechanism SHALL be decoupled from synchronous authorization and clinical transaction execution.

Haven SHALL NOT require a ledger write, Merkle computation, consensus operation, or external anchoring transaction to complete before returning a valid clinical response.

The execution model SHALL therefore be:



This architecture prevents the audit subsystem from becoming a clinical availability dependency.

7.2 Audit Event Envelope

Every consequential authorization or state transition SHALL generate a standardized event envelope.

Representative format:

```
{
  "event_id": "evt_01J...",
  "event_type": "AUTHORIZATION_DECISION",
  "event_version": "1.0",
  "timestamp": "2026-09-03T08:31:14.221Z",
  "request_id": "req_01J...",
  "actor": {
```

```

    "id": "sub_01J...",
    "type": "CLINICIAN"
  },
  "resource": {
    "id": "res_01J...",
    "type": "CLINICAL_RECORD",
    "classification": "SENSITIVE"
  },
  "action": "READ",
  "purpose": "TREATMENT",
  "decision": "ALLOW",
  "operational_state": "NORMAL",
  "policy_version": "haven-policy-2026.09.03",
  "jurisdiction": "NG",
  "attribute_assurance": "VERIFIED"
}

```

The event SHALL be immutable once accepted into the audit pipeline.

7.3 Event Queue

The audit producer SHALL publish events to a durable asynchronous queue or equivalent append-oriented event infrastructure.

The queue SHALL provide:

```

DURABILITY
ORDERING WHERE REQUIRED
RETRY
DEAD-LETTER HANDLING
BACKPRESSURE MANAGEMENT
DELIVERY OBSERVABILITY
INTEGRITY VERIFICATION

```

A temporary audit consumer outage SHALL not prevent ordinary authorization from continuing, subject to defined local durability limits.

Where queue pressure approaches configured thresholds, the system SHALL expose operational telemetry and activate the applicable resilience policy.

7.4 Merkle Batch Formation

Audit events SHALL be cryptographically grouped into batches.

For each event:

```

event_hash = SHA-256(canonical_event)

```

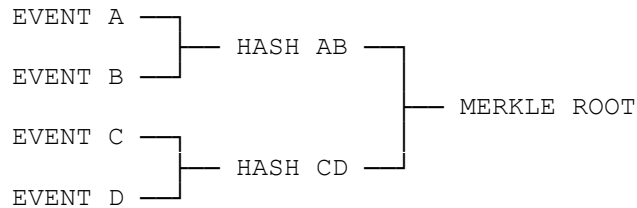
Events within a defined interval SHALL be ordered according to the audit pipeline's canonical ordering rule.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

The system SHALL construct a Merkle tree from the event hashes.

The resulting root SHALL represent the cryptographic state of that audit batch.

Conceptually:



7.5 Scheduled Ledger Anchoring

Ledger state anchoring SHALL occur on a scheduled interval rather than synchronously per request.

A default implementation MAY use a **five-minute anchoring interval**.

```
T+00:00 → Begin Batch
T+00:00-04:59 → Collect Events
T+05:00 → Close Batch
T+05:00 → Compute Merkle Root
T+05:00 → Anchor Root
T+05:00+ → Begin Next Batch
```

The five-minute interval is a configurable operational parameter, not a protocol requirement that must apply identically to every deployment.

Higher-risk environments MAY adopt shorter intervals.

Lower-risk deployments MAY use longer intervals where permitted by policy.

The critical invariant is that the audit chain remains cryptographically verifiable and that the anchoring cadence is itself governed and observable.

7.6 Audit Integrity Model

The audit system SHALL make unauthorized historical modification detectable.

The integrity chain SHALL conceptually be:

```
RAW EVENT
↓
CANONICAL SERIALIZATION
↓
EVENT HASH
```

↓
MERKLE BATCH
↓
MERKLE ROOT
↓
LEDGER ANCHOR
↓
VERIFICATION PROOF

A verifier SHALL be able to establish that a historical event belongs to a specific anchored batch without requiring the original application to be trusted.

This transforms the audit trail from a conventional application log into independently verifiable governance evidence.

7.7 Failure Handling

If ledger anchoring fails:

CLINICAL REQUEST
↓
NOT BLOCKED
↓
AUDIT EVENT RETAINED
↓
BATCH REMAINS PENDING
↓
ANCHOR RETRIED
↓
SUCCESSFUL ANCHOR
↓
STATE MARKED FINAL

The system SHALL NOT discard unanchored events.

Unanchored status SHALL itself be observable.

A pending batch SHALL retain its cryptographic identity so that subsequent anchoring cannot silently alter its contents.

8. IMMUTABLE AUDIT STREAMING FORMAT

8.1 Canonical Event Serialization

For cryptographic integrity to remain deterministic, identical audit events SHALL produce identical canonical representations before hashing.

Haven SHALL therefore define a canonical serialization profile for audit events.

The serialization process SHALL normalize:

- field ordering;
- data types;
- timestamp representation;
- character encoding;
- null handling;
- numerical representation;
- identifier formatting;
- policy version identifiers.

The canonical event SHALL then be hashed.

```
RAW_EVENT
  ↓
SCHEMA_VALIDATION
  ↓
CANONICALIZATION
  ↓
SERIALIZATION
  ↓
SHA-256_HASH
  ↓
AUDIT_STREAM
```

The application SHALL NOT be permitted to alter an event after its canonical hash has been generated.

8.2 Audit Event Classes

The audit stream SHALL distinguish between different categories of governance events.

At minimum:

```
AUTHORIZATION_DECISION
ATTRIBUTE_UPDATE
POLICY_ACTIVATION
POLICY_REVOCATION
CONSENT_CHANGE
```

WORKFLOW_TRANSITION
SYSTEM_STATE_CHANGE
BREAK_GLASS_ACTIVATION
BREAK_GLASS_EXPIRATION
BREAK_GLASS_REVIEW
DATA_ACCESS
DATA_DISCLOSURE
INTEGRATION_EVENT
SECURITY_EVENT
LEDGER_ANCHOR

This classification permits independent analysis of security, compliance, clinical continuity, and system governance activity.

8.3 Event Identity

Every audit event SHALL possess a globally unique event identifier.

The identifier SHALL remain immutable throughout the event lifecycle.

A representative envelope:

```
{  
  "event_id": "evt_01JQ7Y...",  
  "event_type": "DATA_ACCESS",  
  "event_version": "1.0",  
  "schema_id": "haven.audit.v1",  
  "occurred_at": "2026-09-03T08:35:11.821Z",  
  "recorded_at": "2026-09-03T08:35:11.824Z"  
}
```

`occurred_at` identifies when the event happened.

`recorded_at` identifies when the audit infrastructure accepted it.

The distinction is essential for detecting delayed, replayed, or reordered events.

8.4 Previous-Event Linking

Where ordering is required, each event MAY include a cryptographic reference to the preceding event within the relevant stream.

```
{  
  "event_id": "evt_01J...",  
  "previous_event_hash": "sha256:8f4c...",  
  "stream_id": "clinical-governance-ng"  
}
```

This produces a verifiable sequence:

```
EVENT A
  ↓
HASH A
  ↓
EVENT B + HASH A
  ↓
HASH B
  ↓
EVENT C + HASH B
```

Modification, insertion, or deletion within the sequence becomes detectable.

The implementation SHALL avoid assuming that a single globally ordered stream is required for every deployment. Independent streams MAY be used for different tenants, jurisdictions, environments, or governance domains.

9. MERKLE BATCHING & LEDGER STATE ANCHORING

9.1 Batch Identifier

Each audit batch SHALL possess a unique batch identifier.

```
{
  "batch_id": "batch_20260903_0835",
  "stream_id": "clinical-governance-ng",
  "opened_at": "2026-09-03T08:35:00Z",
  "closed_at": "2026-09-03T08:40:00Z",
  "event_count": 1847,
  "merkle_algorithm": "SHA-256",
  "merkle_root": "sha256:...",
  "anchor_status": "PENDING"
}
```

The batch SHALL remain immutable after closure.

9.2 Merkle Proof

A verifier SHALL be capable of proving that a specific event belongs to an anchored batch.

Conceptually:

```
EVENT HASH
  ↓
SIBLING HASH
  ↓
PARENT HASH
  ↓
SIBLING HASH
  ↓
PARENT HASH
  ↓
MERKLE ROOT
```

The verifier SHALL compare the reconstructed root with the anchored root.

If the values differ, the event or its associated proof SHALL be considered inconsistent with the anchored batch.

9.3 Ledger Anchor Object

The ledger anchor SHALL contain the minimum information required to establish the cryptographic state of the batch without unnecessarily exposing protected clinical data.

```
{
  "anchor_id": "anc_01J...",
}
```

```
"batch_id": "batch_20260903_0835",
"stream_id": "clinical-governance-ng",
"merkle_root": "sha256:...",
"event_count": 1847,
"created_at": "2026-09-03T08:40:02Z",
"policy_epoch": "2026.09",
"anchor_status": "FINAL"
}
```

The ledger SHALL store cryptographic evidence rather than raw clinical payloads unless a specific deployment has an independently justified requirement to store additional information.

This preserves the separation between:

clinical data storage

and

governance evidence anchoring.

9.4 Five-Minute Anchoring Cadence

The default reference architecture SHALL support a five-minute audit anchoring cadence.

The cadence SHALL be configurable by deployment policy.

A production scheduler SHALL expose:

```
batch_interval
maximum_batch_size
maximum_batch_age
retry_interval
maximum_retry_attempts
backoff_strategy
anchor_timeout
```

A maximum batch age SHALL exist independently of batch size so that low-volume systems cannot leave audit events indefinitely unanchored.

9.5 Anchor Failure State

The anchoring subsystem SHALL explicitly expose:

```
COLLECTING
SEALED
HASHED
PENDING_ANCHOR
ANCHORING
ANCHORED
ANCHOR_FAILED
```

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

RETRYING
QUARANTINED

A failed anchor SHALL not invalidate the underlying audit events.

The batch SHALL remain cryptographically sealed while the anchoring service retries.

9.6 Audit Replay Protection

The audit consumer SHALL detect duplicate event identifiers.

If the same event is received more than once:

FIRST EVENT
→ ACCEPT

DUPLICATE EVENT
→ DETECT
→ DO NOT CREATE SECOND GOVERNANCE EVENT
→ RECORD DUPLICATION SIGNAL

A duplicate shall not silently overwrite the original event.

The duplicate condition itself MAY become a security telemetry event.

10. POLICY-AS-CODE REGULATORY ABSTRACTION

10.1 Separation of Law and Application Code

Haven SHALL separate legal interpretation from application binaries.

The application SHALL implement stable enforcement interfaces.

Regulatory requirements SHALL be represented through independently versioned declarative policy packages.

The architectural separation is:

```
STATUTE / REGULATION
    ↓
LEGAL INTERPRETATION
    ↓
POLICY REQUIREMENT
    ↓
DECLARATIVE POLICY
    ↓
POLICY COMPILER
    ↓
RUNTIME POLICY PACKAGE
    ↓
ABAC DECISION
    ↓
PEP ENFORCEMENT
```

This architecture prevents a change in regulatory interpretation from requiring a complete rewrite of the core application.

10.2 Nigeria Data Protection Act Policy Pack

For Nigerian deployments, NDPA 2023 requirements SHALL be represented through a jurisdiction-specific policy pack.

The policy pack SHOULD be capable of expressing controls relating to areas such as:

```
PURPOSE LIMITATION
DATA MINIMIZATION
LAWFUL PROCESSING
CONSENT CONDITIONS
DATA SUBJECT RIGHTS
SECURITY SAFEGUARDS
CROSS-BORDER TRANSFER CONDITIONS
RETENTION REQUIREMENTS
PROCESSOR / CONTROLLER RESPONSIBILITIES
BREACH GOVERNANCE
```

The technical system SHALL not claim that a machine-readable rule is itself the law.

Instead, each rule SHALL reference its governing legal or regulatory source and the approved interpretation used to translate that requirement into an executable control.

10.3 Regulatory Policy Object

A policy package SHALL carry governance metadata.

```
{  
  "policy_id": "NG-NDPA-2023",  
  "version": "2026.09.03.1",  
  "jurisdiction": "NG",  
  "framework": "NDPA",  
  "effective_from": "2026-09-03T00:00:00Z",  
  "status": "ACTIVE",  
  "authority": "HAVEN_POLICY_GOVERNANCE",  
  "legal_reference": "NDPA_2023",  
  "compiler_version": "haven-policy-compiler-1.4",  
  "policy_hash": "sha256:...",  
  "signature_status": "VERIFIED"  
}
```

The legal reference SHALL identify the source.

The policy itself SHALL contain the operational interpretation approved for Haven deployment.

10.4 Policy Versioning

No regulatory policy SHALL be modified in place.

A change SHALL generate a new version.

```
POLICY v1  
  ↓  
CHANGE PROPOSED  
  ↓  
LEGAL REVIEW  
  ↓  
TECHNICAL VALIDATION  
  ↓  
POLICY v2  
  ↓  
SIGN  
  ↓  
DISTRIBUTE  
  ↓  
VERIFY  
  ↓  
ACTIVATE
```

Historical decisions SHALL retain the policy version under which they were evaluated.

This is essential for forensic reconstruction.

A decision made under Policy Version 2026.09.03.1 SHALL never be retrospectively represented as though it had been evaluated under a later policy.

10.5 Multi-Jurisdictional Policy Selection

The runtime SHALL select regulatory policies according to jurisdictional context.

Conceptually:

```
REQUEST
  ↓
IDENTIFY DATA JURISDICTION
  ↓
IDENTIFY ACTOR JURISDICTION
  ↓
IDENTIFY PROCESSING JURISDICTION
  ↓
IDENTIFY TRANSFER CONTEXT
  ↓
SELECT APPLICABLE POLICY SET
  ↓
EVALUATE
```

Where multiple regulatory regimes apply simultaneously, the policy engine SHALL evaluate their applicable constraints rather than assuming that one framework automatically supersedes all others.

A deployment MAY therefore maintain policy packs for:

```
NG / NDPA
EU / GDPR
US / HIPAA
UK / UK GDPR
OTHER DEPLOYMENT-SPECIFIC REGIMES
```

The exact applicable policy set SHALL remain deployment- and transaction-dependent.

10.6 Regulatory Change Without Binary Replacement

When a regulatory rule changes, the intended execution model is:

```
LEGAL CHANGE
  ↓
POLICY UPDATE
  ↓
VALIDATION
```

↓
SIGNATURE
↓
POLICY DISTRIBUTION
↓
LOCAL COMPILATION
↓
ATOMIC ACTIVATION

The application binary remains stable.

This reduces the operational cost of regulatory change while preserving an explicit governance boundary between:

software capability

and

regulatory authorization.

11. POLICY DISTRIBUTION & TRUSTED ACTIVATION

11.1 Signed Policy Bundles

Every production policy bundle SHALL be cryptographically signed before distribution.

The runtime SHALL verify:

```
POLICY ID  
VERSION  
SIGNATURE  
SIGNING KEY  
ISSUER  
HASH  
EFFECTIVE DATE  
EXPIRATION / RETIREMENT STATE
```

An invalid signature SHALL prevent activation.

11.2 Atomic Policy Activation

A new policy SHALL first be loaded into an isolated validation environment.

Only after successful validation SHALL it become eligible for activation.

The runtime SHALL support atomic switching:

```
ACTIVE POLICY A  
    ↓  
LOAD POLICY B  
    ↓  
VERIFY B  
    ↓  
COMPILE B  
    ↓  
TEST B  
    ↓  
ACTIVATE B  
    ↓  
RETIRE A
```

At no point should the runtime operate with a partially loaded policy.

11.3 Policy Rollback

Every policy activation SHALL have a rollback mechanism.

Rollback SHALL itself be governed.

A rollback event SHALL record:

```
previous_policy_version
target_policy_version
rollback_reason
authorized_by
timestamp
policy_hash
system_state
```

A rollback SHALL not erase evidence that the superseded policy was previously active.

11.4 Policy Governance Separation

The individual who authors a policy SHALL not automatically possess unilateral authority to activate it in production.

Haven SHOULD implement separation of duties between:

```
POLICY_AUTHOR
POLICY_REVIEWER
POLICY_APPROVER
POLICY_SIGNER
POLICY_DEPLOYER
```

Where organizational scale permits, these roles SHOULD be independently attributable.

This creates a governance barrier against silent policy manipulation.

11.5 Policy Activation Audit

Policy lifecycle events SHALL enter the same asynchronous audit architecture used for authorization events.

At minimum:

```
POLICY_CREATED
POLICY_MODIFIED
POLICY_REVIEWED
POLICY_APPROVED
POLICY_SIGNED
POLICY_DISTRIBUTED
POLICY_ACTIVATED
POLICY_ROLLED_BACK
POLICY_REVOKED
```

Each event SHALL reference the corresponding policy hash.

Consequently, the audit system can establish not merely that a decision occurred, but **which exact policy artifact governed that decision**.

11.6 Developer Implementation Contract

The production implementation SHALL expose policy lifecycle interfaces through a controlled management plane.

Conceptual API:

```
POST    /v1/policies
POST    /v1/policies/{policyId}/validate
POST    /v1/policies/{policyId}/approve
POST    /v1/policies/{policyId}/sign
POST    /v1/policies/{policyId}/activate
POST    /v1/policies/{policyId}/rollback
GET     /v1/policies/{policyId}
GET     /v1/policies/{policyId}/versions
GET     /v1/policies/{policyId}/integrity
```

These management operations SHALL remain outside the clinical request execution path.

11.7 Mermaid Policy Activation Sequence

```
sequenceDiagram
    participant A as Policy Author
    participant G as Governance Plane
    participant V as Validator
    participant S as Signing Service
    participant R as Runtime
    participant L as Audit Queue

    A->>G: Submit Policy Package
    G->>V: Validate Schema + Rules
    V-->>G: Validation Result
    G->>S: Request Signature
    S-->>G: Signed Policy
    G->>R: Distribute Bundle
    R->>R: Verify Signature
    R->>R: Compile Policy
    R->>R: Atomic Activation
    R->>L: Policy Activation Event
```

The runtime SHALL reject any policy package that fails validation, signature verification, compatibility checks, or activation integrity requirements.

12. OPENAPI 3.0 AUTHORIZATION MIDDLEWARE CONTRACT

12.1 Purpose

The Haven authorization layer SHALL expose a stable middleware contract through which protected services can submit authorization contexts and receive deterministic policy decisions.

The interface SHALL remain independent of the underlying policy engine implementation.

OPA/WASM, an equivalent embedded policy runtime, or a future policy execution engine MAY replace the underlying PDP implementation without requiring protected applications to change their authorization contract.

The middleware SHALL therefore operate as a **stable enforcement interface over an independently evolving policy runtime**.

12.2 OpenAPI Definition

The following represents the reference authorization interface:

```
openapi: 3.0.3

info:
  title: Haven Authorization Enforcement API
  version: 1.0.0
  description: >
    Local authorization middleware contract for Haven-protected services.

paths:
  /v1/authorize:
    post:
      operationId: evaluateAuthorization
      summary: Evaluate an authorization request
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/AuthorizationRequest'

      responses:
        '200':
          description: Authorization decision
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/AuthorizationDecision'
```

```

'400':
  description: Invalid authorization context
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'

'403':
  description: Policy denial
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'

'409':
  description: Authorization requires emergency pathway
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/BreakGlassRequired'

'503':
  description: Authorization infrastructure unavailable
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/DegradedStateResponse'

```

components:

schemas:

```

AuthorizationRequest:
  type: object
  required:
    - request_id
    - subject
    - resource
    - action
    - purpose
    - environment
  properties:
    request_id:
      type: string
    subject:
      $ref: '#/components/schemas/Subject'
    resource:
      $ref: '#/components/schemas/Resource'
    action:
      $ref: '#/components/schemas/Action'
    purpose:
      type: string
    environment:
      $ref: '#/components/schemas/Environment'

```

Subject:
 type: object
 required:
 - id
 properties:
 id:
 type: string
 role:
 type: string
 organization:
 type: string
 assurance_level:
 type: string

Resource:
 type: object
 required:
 - id
 - type
 properties:
 id:
 type: string
 type:
 type: string
 classification:
 type: string
 jurisdiction:
 type: string

Action:
 type: object
 required:
 - operation
 properties:
 operation:
 type: string
 scope:
 type: array
 items:
 type: string

Environment:
 type: object
 properties:
 operational_state:
 type: string
 enum:
 - NORMAL
 - DEGRADED
 - EMERGENCY_BREAK_GLASS
 risk_level:
 type: string
 timestamp:
 type: string
 format: date-time

```

AuthorizationDecision:
  type: object
  required:
    - decision
    - decision_id
    - policy_version
  properties:
    decision:
      type: string
      enum:
        - ALLOW
        - DENY
        - ALLOW_WITH_CONSTRAINTS
        - REQUIRE_STEP_UP
        - REQUIRE_BREAK_GLASS
    decision_id:
      type: string
    policy_version:
      type: string
    policy_hash:
      type: string
    constraints:
      type: array
      items:
        type: string
    obligations:
      type: array
      items:
        type: string

```

```

ErrorResponse:
  type: object
  required:
    - error
  properties:
    error:
      type: object
      required:
        - code
        - message
        - request_id
      properties:
        code:
          type: string
        message:
          type: string
        request_id:
          type: string
        policy_version:
          type: string

```

```

BreakGlassRequired:
  type: object
  properties:
    error:
      type: object

```

```

    properties:
      code:
        type: string
        example: EMERGENCY_AUTHORIZATION_REQUIRED
      state:
        type: string
        example: DEGRADED
      break_glass_available:
        type: boolean
        example: true

DegradedStateResponse:
  type: object
  properties:
    error:
      type: object
      properties:
        code:
          type: string
          example: AUTHORIZATION_DEGRADED
        state:
          type: string
          example: DEGRADED

```

12.3 Middleware Execution Requirements

The middleware SHALL perform the following operations:

1. RECEIVE REQUEST
2. VALIDATE SCHEMA
3. NORMALIZE CONTEXT
4. ATTACH LOCAL ATTRIBUTE STATE
5. IDENTIFY ACTIVE POLICY
6. INVOKE IN-MEMORY PDP
7. RETURN DECISION
8. ENFORCE DECISION
9. ASYNCHRONOUSLY EMIT AUDIT EVENT

The middleware SHALL NOT:

```

CALL REMOTE PIP SYNCHRONOUSLY
WRITE TO LEDGER SYNCHRONOUSLY
WAIT FOR MERKLE ANCHORING
REINTERPRET STATUTORY POLICY
OVERRIDE PDP CONSTRAINTS
SILENTLY DISABLE AUDITING

```

12.4 Decision Integrity

The middleware SHALL treat the PDP response as a signed or integrity-protected governance object where the deployment security model requires it.

At minimum, the response SHALL carry:

```
decision_id  
policy_version  
policy_hash  
jurisdiction  
timestamp  
constraints  
obligations
```

The policy hash is particularly important for forensic reconstruction.

Two authorization decisions with identical inputs but different policy hashes are not necessarily equivalent decisions.

13. REGULATORY POLICY SCHEMA

13.1 Declarative Policy Structure

Regulatory policies SHALL be represented in a machine-readable structure independent of application binaries.

A reference policy object:

```
{
  "policy_id": "NG-NDPA-2023",
  "version": "2026.09.03.1",
  "jurisdiction": "NG",
  "framework": "NDPA",
  "status": "ACTIVE",
  "effective_from": "2026-09-03T00:00:00Z",
  "rules": [
    {
      "rule_id": "PURPOSE_LIMITATION",
      "condition": {
        "purpose_declared": true
      },
      "effect": "ALLOW_IF_MATCHED"
    },
    {
      "rule_id": "SECONDARY_USE_RESTRICTION",
      "condition": {
        "secondary_use": true
      },
      "effect": "REQUIRE_ADDITIONAL_AUTHORIZATION"
    }
  ],
  "policy_hash": "sha256:...",
  "signature": {
    "algorithm": "Ed25519",
    "status": "VERIFIED"
  }
}
```

This schema is an implementation representation of governance policy. It SHALL NOT be presented as a literal transcription of statutory text.

13.2 Rule Object

Every rule SHOULD possess:

```
rule_id
description
jurisdiction
legal_reference
effective_date
conditions
```

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

obligations
effect
priority
exceptions
policy_version

The `legal_reference` field establishes traceability between an executable policy and the legal or regulatory requirement from which it was derived.

13.3 Policy Effect Types

The policy engine SHOULD support at least:

ALLOW
DENY
ALLOW_WITH_CONSTRAINTS
REQUIRE_CONSENT
REQUIRE_STEP_UP
REQUIRE_BREAK_GLASS
RESTRICT_SCOPE
REDACT
DEFER

This allows regulatory obligations to be expressed as executable outcomes rather than forcing every requirement into an oversimplified allow/deny model.

13.4 Statutory Interpretation Boundary

Haven SHALL maintain a strict distinction between:

LAW

LEGAL INTERPRETATION

TECHNICAL POLICY

RUNTIME ENFORCEMENT

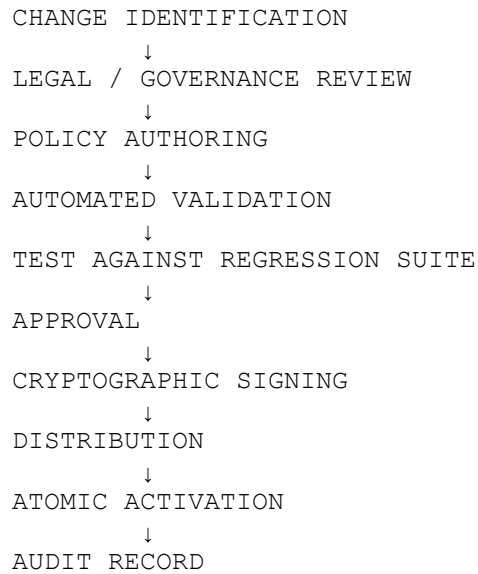
The runtime SHALL execute the approved technical policy.

It SHALL not independently determine what a statute legally means.

This distinction protects the architecture from creating an implicit software-based legal interpretation engine.

13.5 Regulatory Policy Change Control

A regulatory policy change SHALL require:



No policy update SHALL become active merely because it has been uploaded to a policy repository.

14. POLICY REGRESSION & COMPLIANCE TESTING

14.1 Policy Test Suite

Every production regulatory policy package SHALL possess an executable regression suite.

Tests SHALL include:

POSITIVE AUTHORIZATION
NEGATIVE AUTHORIZATION
MISSING ATTRIBUTE
STALE ATTRIBUTE
REVOKED ATTRIBUTE
WRONG JURISDICTION
INVALID PURPOSE
CONSENT FAILURE
EMERGENCY ACCESS
POLICY EXPIRATION
POLICY CONFLICT
POLICY ROLLBACK

A policy bundle that fails mandatory regression tests SHALL not become eligible for production activation.

14.2 Deterministic Policy Testing

Given identical:

SUBJECT
RESOURCE
ACTION
PURPOSE
ENVIRONMENT
ATTRIBUTES
POLICY VERSION

the PDP SHALL produce a deterministic decision.

The test harness SHALL therefore support replay of historical authorization contexts.

This capability is critical for forensic analysis.

An investigator should be able to reconstruct the policy decision using the historical policy version and recorded authorization context without requiring the original application instance to still exist.

14.3 Compliance Simulation

Before activation, policy changes SHOULD be tested against historical decision samples.

Conceptually:

```
NEW POLICY
  ↓
HISTORICAL REQUEST SET
  ↓
RE-EVALUATION
  ↓
COMPARE OLD vs NEW DECISIONS
  ↓
IDENTIFY BEHAVIOR CHANGES
  ↓
GOVERNANCE REVIEW
```

This creates a controlled mechanism for detecting unintended authorization changes before they affect production workflows.

14.4 Policy Drift Detection

Runtime policy state SHALL be continuously compared against the expected signed policy state.

Drift conditions include:

```
UNEXPECTED POLICY HASH
UNKNOWN POLICY VERSION
UNAUTHORIZED POLICY ACTIVATION
MISSING POLICY
EXPIRED POLICY
LOCAL POLICY DIFFERENCE
SIGNATURE FAILURE
```

A detected drift SHALL generate an audit and security event.

The system SHALL not silently normalize unauthorized drift.

15. COMPLIANCE CONTROL PLANE

15.1 Compliance as Executable Infrastructure

Haven SHALL treat compliance controls as infrastructure primitives rather than documentation-only requirements.

The compliance control plane SHALL connect:

```
REGULATORY POLICY
  ↓
ATTRIBUTE REQUIREMENTS
  ↓
AUTHORIZATION DECISIONS
  ↓
WORKFLOW CONSTRAINTS
  ↓
AUDIT EVIDENCE
  ↓
COMPLIANCE TELEMETRY
  ↓
GOVERNANCE REVIEW
```

The objective is not to automate legal judgment.

The objective is to ensure that approved governance requirements can be translated into repeatable technical controls.

15.2 Compliance Control Object

A compliance control SHALL contain:

```
{
  "control_id": "NG-DATA-PURPOSE-001",
  "framework": "NDPA",
  "jurisdiction": "NG",
  "requirement_type": "PURPOSE_LIMITATION",
  "policy_version": "2026.09.03.1",
  "control": {
    "condition": "declared_purpose_matches_authorized_purpose",
    "effect": "ALLOW"
  },
  "evidence": {
    "required": true,
    "event_type": "AUTHORIZATION_DECISION"
  },
  "owner": "HAVEN_GOVERNANCE",
  "status": "ACTIVE"
}
```

The control object links a governance requirement to its runtime enforcement mechanism and evidentiary output.

15.3 Evidence Mapping

For each material compliance control, Haven SHOULD be capable of answering:

```
WHAT REQUIREMENT?  
  ↓  
WHAT POLICY?  
  ↓  
WHAT VERSION?  
  ↓  
WHAT DECISION?  
  ↓  
WHAT ACTOR?  
  ↓  
WHAT RESOURCE?  
  ↓  
WHAT SYSTEM STATE?  
  ↓  
WHAT AUDIT EVENT?  
  ↓  
WHAT MERKLE BATCH?  
  ↓  
WHAT LEDGER ANCHOR?
```

This creates an evidence chain from regulatory requirement to cryptographically anchored system activity.

15.4 Continuous Compliance State

The compliance plane SHALL expose machine-readable control states.

```
COMPLIANT  
NON_COMPLIANT  
DEGRADED  
CONTROL_PENDING  
CONTROL_FAILED  
POLICY_EXPIRED  
EVIDENCE_PENDING  
EVIDENCE_ANCHORED  
REVIEW_REQUIRED
```

A `COMPLIANT` state SHALL represent compliance against the defined technical control set at the evaluated point in time.

It SHALL NOT be interpreted as an absolute legal certification.

15.5 Compliance Evidence Without Clinical Data Replication

The compliance architecture SHALL avoid unnecessarily duplicating protected clinical information into governance systems.

Where possible, evidence SHALL reference:

RESOURCE IDENTIFIER
EVENT IDENTIFIER
POLICY IDENTIFIER
POLICY HASH
DECISION
TIMESTAMP
JURISDICTION
MERKLE PROOF
LEDGER ANCHOR

rather than reproducing the underlying clinical payload.

This supports a **minimum-evidence architecture** in which governance can be demonstrated without creating a second uncontrolled repository of sensitive health information.

15.6 Regulatory Enforcement Moat

The long-term architectural moat emerges from the accumulation of interoperable governance infrastructure.

As Haven integrations mature, each deployment can contribute to a structured governance environment containing:

VERSIONED POLICY PACKS
JURISDICTIONAL MAPPINGS
CONTROL LIBRARIES
AUDIT SCHEMAS
ATTRIBUTE TAXONOMIES
WORKFLOW CONSTRAINTS
EMERGENCY GOVERNANCE PATTERNS
COMPLIANCE TEST SUITES
FORENSIC RECONSTRUCTION CAPABILITY

The resulting system becomes more valuable as more regulated workflows depend upon its governance layer.

However, the moat SHALL be based on **technical interoperability, operational evidence, policy infrastructure, and switching costs**, not on intentionally obstructing lawful competition or preventing customers from exercising legitimate control over their own systems and data.

16. ZERO-TRUST SERVICE-TO-SERVICE GOVERNANCE

16.1 Service Identity

Haven SHALL treat every service, integration, adapter, worker, and execution environment as an independently governed security principal.

A service SHALL NOT inherit authorization solely because it operates within a trusted institutional network.

Each service identity SHALL possess:

```
service_id
organization_id
environment
deployment_id
trust_level
credential_state
policy_scope
jurisdiction
issued_at
expires_at
```

Service authentication establishes **who is making the request**.

ABAC establishes **whether that actor is permitted to perform the requested operation**.

These functions SHALL remain logically distinct.

16.2 Workload Authorization

Service-to-service requests SHALL pass through the same enforcement architecture as human-originated requests where the requested operation carries protected or consequential authority.

The authorization context SHALL identify:

```
HUMAN ACTOR
    ↓
ORIGINATING APPLICATION
    ↓
SERVICE IDENTITY
    ↓
REQUESTED OPERATION
    ↓
RESOURCE
    ↓
PURPOSE
    ↓
JURISDICTION
    ↓
```

POLICY

Where a service acts on behalf of a human actor, the system SHALL preserve the relationship between the initiating actor and the executing workload.

The service SHALL not become an opaque authority substitution.

16.3 Delegated Authority

Delegation SHALL be explicit.

A service operating under delegated authority SHALL carry a bounded delegation context:

```
{
  "delegation_id": "del_01J...",
  "principal_id": "clinician_01J...",
  "delegate_id": "service_01J...",
  "scope": [
    "READ_CLINICAL_RECORD"
  ],
  "purpose": "TREATMENT",
  "issued_at": "2026-09-03T08:45:00Z",
  "expires_at": "2026-09-03T09:00:00Z",
  "policy_version": "haven-policy-2026.09.03"
}
```

The delegated service SHALL possess no greater authority than the authority explicitly delegated to it.

This SHALL be evaluated as an authorization constraint rather than assumed by the application.

16.4 Trust Boundary Crossing

Every transition between organizational or technical trust domains SHALL constitute a policy evaluation boundary.

Examples include:

```
HOSPITAL → HAVEN
LABORATORY → HAVEN
PHARMACY → HAVEN
HMO → HAVEN
HAVEN → EXTERNAL SERVICE
INSTITUTION A → INSTITUTION B
NATIONAL SYSTEM → PRIVATE SYSTEM
```

The receiving environment SHALL not assume that upstream authorization remains valid merely because the upstream system has already evaluated the request.

A new enforcement point SHALL evaluate the attributes relevant to its own trust boundary.

16.5 Anti-Confused-Deputy Control

Haven SHALL protect against confused-deputy conditions in which a privileged service is manipulated into performing an operation that the initiating actor was not authorized to perform.

The policy engine SHOULD evaluate:

```
initiating_principal  
delegated_principal  
service_identity  
requested_resource  
requested_action  
delegation_scope  
purpose  
policy_version
```

The authorization decision SHALL be constrained by the narrowest applicable authority.

Conceptually:

```
EFFECTIVE AUTHORITY  
=  
PRINCIPAL AUTHORITY  
∩  
DELEGATED AUTHORITY  
∩  
SERVICE AUTHORITY  
∩  
RESOURCE POLICY  
∩  
REGULATORY POLICY
```

No component SHALL enlarge authority merely by forwarding a request through a more privileged service.

17. ZERO-TRUST DATA ACCESS CONTROLS

17.1 Resource Classification

Resources SHALL possess machine-readable classification attributes.

A reference classification model:

PUBLIC
INTERNAL
CONFIDENTIAL
SENSITIVE
HIGHLY_SENSITIVE
CLINICAL_CRITICAL
REGULATED

The classification SHALL influence policy evaluation.

A clinician's ability to access one class of information SHALL not automatically grant access to every resource belonging to the same patient or institution.

17.2 Purpose-Bound Access

The purpose of access SHALL be an explicit authorization attribute for regulated resources.

Examples:

TREATMENT
CARE_COORDINATION
PAYMENT
OPERATIONS
PUBLIC_HEALTH
RESEARCH
AUDIT
LEGAL_COMPLIANCE
EMERGENCY_TREATMENT

The PDP SHALL compare the declared purpose with the permitted purposes for the requested resource.

A valid identity with an invalid purpose SHALL not receive unrestricted access.

17.3 Scope Restriction

Where complete denial is unnecessary, Haven SHALL support scope-restricted authorization.

For example:

```
{
  "decision": "ALLOW_WITH_CONSTRAINTS",
  "constraints": [
    {
      "type": "FIELD_SCOPE",
      "allowed": [
        "ALLERGIES",
        "CURRENT_MEDICATIONS"
      ]
    },
    {
      "type": "PURPOSE",
      "value": "EMERGENCY_TREATMENT"
    },
    {
      "type": "TIME_LIMIT",
      "expires_at": "2026-09-03T08:55:00Z"
    }
  ]
}
```

This allows the policy engine to enforce the principle of least privilege without forcing every sensitive workflow into a binary allow/deny decision.

17.4 Data Minimization at Enforcement

Where a request requires only a subset of a protected resource, the PEP SHOULD enforce field or object-level restrictions.

The architecture SHALL prefer:

```
REQUEST MINIMUM DATA
    ↓
AUTHORIZE MINIMUM SCOPE
    ↓
RETURN MINIMUM REQUIRED DATA
```

over:

```
AUTHORIZE FULL RESOURCE
    ↓
APPLICATION DECIDES WHAT TO IGNORE
```

The latter model creates unnecessary dependence upon application developers correctly implementing regulatory restrictions.

Haven's enforcement layer SHOULD therefore make excessive access technically harder to perform.

18. COMPLIANCE CONTROL ENFORCEMENT PIPELINE

18.1 Pre-Execution Controls

Before a protected operation executes, the enforcement layer SHALL evaluate:

IDENTITY
ATTRIBUTE ASSURANCE
RESOURCE CLASSIFICATION
PURPOSE
CONSENT STATE
JURISDICTION
POLICY VERSION
OPERATIONAL STATE
CLINICAL CRITICALITY

Only after applicable controls have resolved SHALL the PEP permit execution.

18.2 During-Execution Controls

For long-running workflows, authorization SHALL not necessarily be considered permanently valid merely because it was granted at workflow initiation.

Where policy requires, the system SHOULD support authorization checkpoints during execution.

Examples include:

LONG-RUNNING DATA EXPORT
BULK RECORD ACCESS
CROSS-INSTITUTIONAL WORKFLOW
HIGH-RISK ADMINISTRATIVE ACTION
PRIVILEGED DATA TRANSFER
EMERGENCY SESSION

A material change in risk, purpose, scope, jurisdiction, or authority MAY trigger re-evaluation.

18.3 Post-Execution Controls

After execution, the audit system SHALL record sufficient evidence to establish what occurred.

The post-execution event SHOULD contain:

request_id
decision_id
event_id
actor
service
resource
action

purpose
policy_version
decision
operational_state
execution_result
timestamp

The execution result SHALL not replace the authorization decision.

A request may be authorized but fail technically.

Conversely, a technical operation succeeding does not establish that it was authorized.

These are separate governance facts.

18.4 Compliance Evidence Correlation

Haven SHALL support correlation between:

```
AUTHORIZATION DECISION
      ↓
APPLICATION REQUEST
      ↓
WORKFLOW EVENT
      ↓
DATA ACCESS EVENT
      ↓
AUDIT EVENT
      ↓
MERKLE BATCH
      ↓
LEDGER ANCHOR
```

The correlation identifier SHALL remain stable across the relevant event chain.

This enables investigators to reconstruct an entire governance sequence without requiring access to every underlying application log.

19. SECURITY EVENT ESCALATION

19.1 Security Signal Classification

Security and governance events SHALL be classified according to severity.

INFO
NOTICE
WARNING
HIGH
CRITICAL

Examples of CRITICAL events may include:

UNAUTHORIZED POLICY MODIFICATION
SIGNATURE VERIFICATION FAILURE
REPEATED BREAK-GLASS ABUSE
ATTRIBUTE REVOCATION BYPASS
AUDIT INTEGRITY FAILURE
UNEXPECTED POLICY DRIFT

The exact severity mapping SHALL be configurable by deployment policy.

19.2 Security Event Object

```
{  
  "event_id": "sec_01J...",  
  "event_type": "POLICY_INTEGRITY_FAILURE",  
  "severity": "CRITICAL",  
  "detected_at": "2026-09-03T08:47:13Z",  
  "service_id": "service_01J...",  
  "policy_id": "NG-NDPA-2023",  
  "expected_hash": "sha256:...",  
  "observed_hash": "sha256:...",  
  "action": "POLICY_ACTIVATION_BLOCKED",  
  "operational_state": "DEGRADED"  
}
```

Security events SHALL enter the asynchronous audit pipeline while simultaneously being made available to the security monitoring plane.

19.3 Escalation Without Clinical Deadlock

Security escalation SHALL not automatically terminate critical clinical workflows.

Where an event represents a security concern but the workflow is clinically critical, the state engine SHALL evaluate whether:

NORMAL CONTINUATION
OR
RESTRICTED CONTINUATION
OR
BREAK-GLASS

is appropriate.

The objective is to prevent security controls from becoming an accidental source of patient-care failure while maintaining complete accountability for exceptional access.

20. POLICY ENFORCEMENT INTEGRITY

20.1 Fail-Safe Policy Loading

The runtime SHALL maintain a verified active policy set.

If a new policy cannot be verified, the runtime SHALL retain the last known valid policy subject to its configured validity period.

It SHALL NOT activate an unverified policy merely because the previous policy is inconvenient or outdated.

The resulting state SHALL be observable:

```
ACTIVE_VALID_POLICY
  ↓
UPDATE RECEIVED
  ↓
VERIFY
  ↓
SUCCESS → ACTIVATE
FAILURE → RETAIN VALID STATE
```

20.2 Policy Expiration

Policies MAY possess explicit expiration or review dates.

An expired policy SHALL not silently continue indefinitely.

Instead, the runtime SHALL evaluate the policy lifecycle rule:

```
ACTIVE
  ↓
REVIEW DUE
  ↓
RENEWED
  OR
SUPERSEDED
  OR
EXPIRED
```

Expiration behavior SHALL be defined by the governing policy and deployment context.

For critical clinical environments, expiration handling SHALL include an explicit continuity pathway rather than an undocumented global shutdown.

20.3 Integrity Failure

If the runtime detects:

HASH MISMATCH
SIGNATURE FAILURE
UNKNOWN SIGNER
CORRUPTED POLICY
UNAUTHORIZED MODIFICATION

the affected policy SHALL be rejected.

The event SHALL generate a security audit record.

Where a valid prior policy remains available, the runtime SHALL preserve it according to policy.

Where no valid policy exists, the operational state engine SHALL determine whether restricted operation, degraded operation, or emergency governance is applicable.

21. ABAC POLICY LOGIC EXAMPLES

21.1 Standard Clinical Access

```
IF
    subject.role = "CLINICIAN"
AND subject.organization = resource.organization
AND purpose = "TREATMENT"
AND resource.classification = "CLINICAL"
AND subject.assurance >= REQUIRED_ASSURANCE
AND jurisdiction_policy = "PERMITTED"

THEN
    ALLOW
```

The actual production policy SHALL include the complete attribute and regulatory conditions relevant to the deployment.

21.2 Restricted Secondary Use

```
IF
    purpose = "RESEARCH"
AND resource.classification = "HIGHLY_SENSITIVE"

THEN
    REQUIRE_ADDITIONAL_POLICY
```

The subsequent policy MAY require:

```
consent
de-identification
approved research purpose
institutional authorization
jurisdictional compatibility
additional audit controls
```

21.3 Emergency Access

```
IF
    operational_state = "DEGRADED"
AND clinical_criticality = "CRITICAL"
AND emergency_declared = true
AND actor.emergency_authority = "VALID"

THEN
    ALLOW_WITH_CONSTRAINTS
```

The resulting access SHALL be limited by scope and time.

22. GOVERNANCE TELEMETRY

22.1 Required Metrics

Production deployments SHALL expose governance metrics including:

```
authorization_latency_p50
authorization_latency_p95
authorization_latency_p99
policy_evaluation_errors
attribute_cache_hit_rate
attribute_staleness_rate
break_glass_activation_count
break_glass_expiration_count
policy_activation_count
policy_rollback_count
policy_drift_events
audit_queue_depth
audit_processing_latency
unanchored_batch_age
ledger_anchor_failures
```

The authorization latency target SHALL remain aligned with the architectural requirement of **less than 10 ms authorization overhead**.

22.2 Governance SLOs

The deployment SHOULD define service-level objectives for:

```
AUTHORIZATION AVAILABILITY
POLICY PROPAGATION
ATTRIBUTE FRESHNESS
AUDIT QUEUE DURABILITY
AUDIT ANCHORING
BREAK-GLASS AVAILABILITY
POLICY INTEGRITY
```

These SLOs SHALL be treated as governance infrastructure objectives rather than ordinary application performance metrics.

22.3 Operational Dashboard

A production governance dashboard SHOULD provide real-time visibility into:

```
ACTIVE POLICY VERSION
POLICY HASH
CURRENT OPERATIONAL STATE
ATTRIBUTE CACHE HEALTH
AUTHORIZATION LATENCY
DENIAL RATE
```

BREAK-GLASS ACTIVITY
AUDIT QUEUE STATUS
PENDING MERKLE BATCHES
ANCHOR STATUS
POLICY DRIFT
SECURITY EVENTS

The dashboard SHALL not expose protected clinical information merely for operational convenience.

23. GOVERNANCE BOUNDARY PRINCIPLE

The Haven architecture SHALL preserve a strict boundary between **technical enforcement** and **legal authority**.

Haven can technically enforce a configured policy.

It cannot, by software alone, declare that a legal interpretation is universally correct.

Accordingly:

LAW
≠
POLICY CONFIGURATION
≠
APPLICATION CODE
≠
RUNTIME DECISION
≠
COMPLIANCE CERTIFICATION

Each layer SHALL retain its own provenance.

This prevents the technical architecture from creating false legal certainty while still allowing approved regulatory requirements to become continuously executable controls.

24. COMPLIANCE POLICY CONTROL MATRIX

The Haven compliance architecture converts regulatory and organizational requirements into executable technical controls without embedding statutory interpretation directly into application binaries.

Each control must establish a deterministic relationship between:

Regulatory Domain → Policy Rule → Required Attributes → Enforcement Point → Decision → Evidence → Operational State

This creates a machine-verifiable compliance chain in which every consequential access decision can be reconstructed without requiring the application itself to interpret legislation.

24.1 Control Matrix

Control Domain	Required ABAC Context	PEP Enforcement	Audit Evidence	Degraded-State Behaviour
Identity Assurance	subject identity, assurance level, credential state	Permit / Deny / Step-Up	identity assurance + decision	Cached identity permitted only within TTL
Purpose Limitation	declared purpose, workflow purpose	Restrict action to declared purpose	purpose + policy version	Cached purpose cannot exceed original scope
Data Minimization	resource classification, requested fields	Field-level restriction	requested vs released scope	Minimum cached scope only
Organizational Boundary	organization_id, tenant_id	Cross-tenant denial or controlled delegation	source + destination organization	Cross-boundary access restricted
Jurisdiction	jurisdiction, processing location	Select jurisdictional policy	jurisdiction + policy bundle	Last valid jurisdiction policy may be used within TTL
Consent / Authorization	consent state, authorization basis	Allow / Restrict / Deny	consent reference + decision	Stale consent cannot silently expand access
Clinical Criticality	workflow criticality, patient-care state	Normal / Restricted / Break-Glass	criticality + state	Emergency pathway remains available
Service Identity	service_id, deployment_id, credential state	Service-to-service authorization	workload identity + decision	Cached service identity only within TTL

Control Domain	Required ABAC Context	PEP Enforcement	Audit Evidence	Degraded-State Behaviour
Data Disclosure	recipient, purpose, destination	Restrict / Redact / Deny	disclosure context	No expansion beyond cached policy scope
Retention	record class, retention state	Prevent unauthorized persistence	lifecycle decision	Local deletion controls remain enforced
Emergency Access	emergency declaration, authority, reason, expiry	Break-glass constraints	complete emergency evidence	Explicit emergency pathway
Policy Integrity	policy hash, signer, version	Reject invalid policy	integrity failure event	Retain last valid policy

24.2 Control Evaluation Contract

Every control must produce a structured result rather than an implicit Boolean.

```
{
  "control_id": "CTRL-DATA-PURPOSE-001",
  "policy_id": "HAVEN-NG-DATA-PURPOSE",
  "policy_version": "2026.09.01",
  "evaluation": "PASS",
  "effect": "ALLOW_WITH_CONSTRAINTS",
  "constraints": {
    "purpose": "CARE_COORDINATION",
    "field_scope": [
      "patient_reference",
      "care_state",
      "care_provider"
    ],
    "expires_at": "2026-09-03T10:30:00Z"
  },
  "evidence_required": true
}
```

The control result becomes part of the authorization evidence envelope and is subsequently streamed into the asynchronous audit system.

24.3 Control Precedence

Where multiple controls apply simultaneously, Haven must resolve them through deterministic precedence rather than application-specific interpretation.

The governing principle is:

```
SAFETY CONSTRAINT
>
REGULATORY CONSTRAINT
>
DATA-SUBJECT CONSTRAINT
>
ORGANIZATIONAL CONSTRAINT
>
WORKFLOW CONSTRAINT
>
RESOURCE PERMISSION
>
DEFAULT APPLICATION PERMISSION
```

A lower-priority rule may never expand authority beyond the maximum authority permitted by a higher-priority rule.

24.4 Compliance Control Lifecycle

Every compliance control exists as a versioned object:

```
DRAFT
↓
VALIDATION
↓
LEGAL / GOVERNANCE REVIEW
↓
SECURITY REVIEW
↓
TESTED
↓
SIGNED
↓
ACTIVATED
↓
MONITORED
↓
REVIEW DUE
├── RENEW
├── SUPERSEDE
└── RETIRE
```

No control should transition directly from an unverified draft into production execution.

25. NDPA 2023 TECHNICAL CONTROL MAPPING

Haven should treat the **Nigeria Data Protection Act 2023 (NDPA)** as a jurisdictional source for policy requirements that must be translated into machine-executable controls.

The architecture does **not** attempt to convert statutory language directly into application code.

Instead:

```
NDPA / APPLICABLE REGULATION
    ↓
LEGAL INTERPRETATION
    ↓
TECHNICAL CONTROL DEFINITION
    ↓
DECLARATIVE POLICY
    ↓
POLICY VALIDATION
    ↓
SIGNED POLICY BUNDLE
    ↓
LOCAL PDP RUNTIME
    ↓
PEP ENFORCEMENT
    ↓
IMMUTABLE EVIDENCE
```

This separation allows the legal interpretation layer to evolve without requiring a rewrite of the core Haven execution engine.

25.1 NDPA-Derived Control Domains

The initial Nigerian policy pack should support technical mappings for applicable domains including:

1. **Lawful and controlled processing**
2. **Purpose limitation**
3. **Data minimization**
4. **Data subject rights**
5. **Security safeguards**
6. **Controller / processor governance**
7. **Data disclosure restrictions**
8. **Cross-border processing controls**
9. **Retention and lifecycle governance**
10. **Personal-data incident governance**
11. **Accountability and evidence generation**
12. **Privacy governance and organizational controls**

These mappings are **technical implementation controls**, not automated legal conclusions. Final interpretation of statutory obligations remains subject to qualified legal and regulatory review.

25.2 Jurisdiction Policy Object

```
{
  "jurisdiction": "NG",
  "framework": "NDPA",
  "policy_pack_id": "HAVEN-NG-PRIVACY",
  "version": "2026.09.01",
  "effective_from": "2026-09-01T00:00:00Z",
  "policy_hash": "sha256:<canonical-policy-hash>",
  "signing_authority": "HAVEN-POLICY-AUTHORITY",
  "status": "ACTIVE",
  "legal_review_status": "APPROVED_FOR_TECHNICAL_IMPLEMENTATION"
}
```

The runtime consumes the **technical policy object**, not the statute itself.

25.3 Cross-Jurisdiction Policy Selection

Where a transaction crosses jurisdictional boundaries, the jurisdiction resolver must identify every applicable policy domain before authorization.

```
REQUEST
  ↓
SUBJECT JURISDICTION
+
RESOURCE JURISDICTION
+
PROCESSING JURISDICTION
+
DESTINATION JURISDICTION
  ↓
JURISDICTION SET
  ↓
POLICY INTERSECTION
  ↓
MOST RESTRICTIVE APPLICABLE CONSTRAINT
  ↓
PDP DECISION
```

The purpose is not to create a universal legal engine.

The purpose is to ensure that Haven never silently executes an action under a policy context that is narrower than the jurisdictional constraints already identified by the platform.

26. PRIVACY-BY-ARCHITECTURE & DATA-SUBJECT RIGHTS EXECUTION

Privacy controls must exist at the enforcement boundary rather than exclusively inside administrative dashboards or retrospective compliance processes.

Haven therefore treats privacy as an execution property.

A request involving regulated personal data must carry sufficient context to determine:

WHO is requesting?
WHAT resource is involved?
WHY is it being accessed?
WHICH jurisdiction applies?
WHAT authority permits the action?
WHAT scope is actually required?
HOW LONG should that authority remain valid?
WHAT evidence must be generated?

26.1 Data-Subject Rights as Policy-Controlled Workflows

Applicable data-subject rights should be implemented as controlled workflow operations rather than direct database commands.

The execution model is:

```
RIGHTS REQUEST
  ↓
IDENTITY / REQUEST VALIDATION
  ↓
SUBJECT RECORD RESOLUTION
  ↓
JURISDICTION + POLICY SELECTION
  ↓
LEGAL / ORGANIZATIONAL EXCEPTION CHECK
  ↓
WORKFLOW AUTHORIZATION
  ↓
CONTROLLED DATA OPERATION
  ↓
RESULT VALIDATION
  ↓
ASYNC AUDIT EVIDENCE
```

Potential workflow classes include:

- ACCESS
- CORRECTION
- ERASURE / DELETION WHERE APPLICABLE
- RESTRICTION WHERE APPLICABLE

- OBJECTION WHERE APPLICABLE
- PORTABILITY WHERE APPLICABLE
- DISCLOSURE HISTORY
- PROCESSING-PREFERENCE CHANGE

The exact availability and scope of each workflow must remain configurable by jurisdictional policy.

26.2 Privacy Request Authorization Object

```
{
  "request_id": "DSR-01J...",
  "subject_reference": "pseudonymous-subject-reference",
  "request_type": "ACCESS",
  "jurisdiction": "NG",
  "requested_scope": {
    "systems": [
      "clinical-records",
      "care-coordination"
    ],
    "data_classes": [
      "PERSONAL_DATA"
    ]
  },
  "purpose": "DATA_SUBJECT_REQUEST",
  "authorization_context": {
    "requester_assurance": "HIGH",
    "identity_verified": true
  },
  "policy_version": "2026.09.01"
}
```

No direct database privilege should be granted merely because a rights request has been opened.

The request itself must pass through the same governance architecture governing every other consequential action.

26.3 Privacy Enforcement States

Privacy operations must also respect the three-tier operational state engine.

NORMAL

Full policy evaluation using current attributes and active policy bundles.

DEGRADED

Use only encrypted, integrity-verified attributes that remain within their approved TTL. No stale attribute may expand the requested scope.

EMERGENCY BREAK-GLASS

Emergency authority may be used only where the applicable policy explicitly permits emergency processing. Emergency authority must remain bounded by purpose, scope, duration and post-event review.

A degraded system must never reinterpret unavailable information as authorization.

27. COMPLIANCE EVALUATION API

The compliance control plane should expose a separate evaluation contract from the authorization endpoint so that governance systems can validate proposed actions without granting execution authority.

27.1 OpenAPI 3.0 Compliance Evaluation Schema

```
openapi: 3.0.3
info:
  title: Haven Compliance Evaluation API
  version: 1.0.0

paths:
  /v1/compliance/evaluate:
    post:
      summary: Evaluate an action against active compliance controls
      operationId: evaluateCompliance
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ComplianceEvaluationRequest'
      responses:
        '200':
          description: Compliance evaluation result
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ComplianceEvaluationResponse'
        '409':
          description: Policy or control conflict
        '422':
          description: Invalid compliance context

components:
  schemas:
    ComplianceEvaluationRequest:
      type: object
      required:
        - subject
        - resource
        - action
        - purpose
        - jurisdiction
      properties:
        subject:
          type: object
        resource:
          type: object
        action:
          type: string
```

```

    purpose:
      type: string
    jurisdiction:
      type: array
      items:
        type: string
    workflow_state:
      type: string
    policy_version:
      type: string

ComplianceEvaluationResponse:
  type: object
  required:
    - evaluation_id
    - status
    - controls
  properties:
    evaluation_id:
      type: string
    status:
      type: string
      enum:
        - COMPLIANT
        - NON_COMPLIANT
        - DEGRADED
        - REVIEW_REQUIRED
    controls:
      type: array
      items:
        $ref: '#/components/schemas/ControlResult'
    policy_hash:
      type: string

ControlResult:
  type: object
  required:
    - control_id
    - result
  properties:
    control_id:
      type: string
    result:
      type: string
      enum:
        - PASS
        - FAIL
        - NOT_APPLICABLE
        - INDETERMINATE
    effect:
      type: string
    evidence_required:
      type: boolean

```

The compliance endpoint is advisory to governance systems unless explicitly integrated with a PEP. It must not become a parallel authorization mechanism capable of producing a conflicting execution decision.

28. REGULATORY DECISION & EVIDENCE SEQUENCE

The following sequence defines the intended production execution path.

```
sequenceDiagram
    participant C as Client
    participant PEP as Policy Enforcement Point
    participant JR as Jurisdiction Resolver
    participant PDP as In-Memory PDP
    participant Q as Audit Queue
    participant M as Merkle Batcher
    participant L as Ledger Anchor

    C->>PEP: Request + Authorization Context
    PEP->>JR: Resolve Applicable Jurisdictions
    JR-->>PEP: Jurisdiction Set
    PEP->>PDP: Normalized ABAC Context
    PDP-->>PEP: Decision + Constraints
    PEP-->>C: Allow / Deny / Constrained Decision

    PEP-)Q: Emit Authorization Evidence
    Q-)M: Deliver Canonical Events
    M->>M: Build Merkle Batch
    M->>L: Submit Merkle Root
    L-->>M: Anchor Confirmation
    M-)Q: Anchor Status / Proof Metadata
```

The critical architectural property is the separation between **clinical execution latency** and **cryptographic evidence finality**.

The authorization response must not wait for:

- queue confirmation,
- Merkle construction,
- ledger consensus,
- external anchoring,
- audit-chain finality.

The synchronous path determines whether the action is permitted.

The asynchronous path proves that the decision occurred under a specific policy, attribute state and operational condition.

28.1 Regulatory Decision Evidence Chain

Every regulated decision should ultimately be reconstructable through:

```
REQUEST_ID
↓
AUTHORIZATION_DECISION_ID
```

↓
POLICY_ID + POLICY_VERSION + POLICY_HASH
↓
ATTRIBUTE_SNAPSHOT / ATTRIBUTE_REFERENCES
↓
JURISDICTION SET
↓
CONTROL RESULTS
↓
PEP ENFORCEMENT RESULT
↓
AUDIT EVENT
↓
EVENT HASH
↓
MERKLE BATCH
↓
MERKLE ROOT
↓
LEDGER ANCHOR
↓
VERIFICATION PROOF

This creates a durable compliance evidence architecture without requiring the ledger to become the clinical system of record.

28.2 Architectural Compliance Moat

The resulting moat is not created by hiding legal requirements inside proprietary code.

It is created by accumulating a technically coherent governance system consisting of:

- versioned policy packs;
- jurisdiction-aware ABAC logic;
- reusable attribute taxonomies;
- standardized PEP contracts;
- deterministic compliance controls;
- encrypted degradation mechanisms;
- emergency governance patterns;
- canonical audit schemas;
- cryptographic evidence chains;
- policy regression suites;
- operational compliance telemetry;
- reproducible forensic verification.

A competing implementation may reproduce individual components.

Reproducing the **integrated governance substrate, validated policy history, operational evidence model and production-tested control library** is substantially more difficult.

Haven therefore converts compliance from a documentation exercise into an executable infrastructure layer.

29. POLICY DISTRIBUTION, VERSION CONTROL & RUNTIME CONSISTENCY

A zero-trust authorization architecture is only as reliable as the policy state being executed.

Haven must therefore treat policy distribution as a controlled software supply chain. A policy cannot become authoritative merely because it exists in a repository, has been downloaded by a service, or has been referenced by an administrator.

The production runtime must establish that a policy is:

1. structurally valid;
2. cryptographically intact;
3. signed by an authorized policy authority;
4. within its validity period;
5. applicable to the target jurisdiction and environment;
6. compatible with the active runtime;
7. successfully evaluated against regression tests;
8. atomically activated.

29.1 Policy Bundle Structure

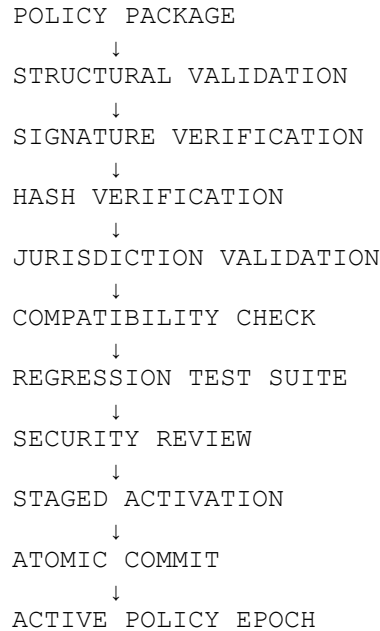
```
{
  "bundle_id": "HAVEN-POLICY-BUNDLE-NG-2026-09",
  "jurisdiction": ["NG"],
  "environment": "PRODUCTION",
  "version": "2026.09.01",
  "issued_at": "2026-09-01T00:00:00Z",
  "effective_from": "2026-09-01T00:00:00Z",
  "expires_at": "2026-12-01T00:00:00Z",
  "policy_hash": "sha256:<bundle-hash>",
  "compiler_version": "haven-policy-compiler-1.x",
  "runtime_compatibility": {
    "minimum": "1.0.0",
    "maximum": "2.x"
  },
  "signature": {
    "algorithm": "approved-signature-algorithm",
    "key_id": "policy-signing-key-01",
    "value": "<signature>"
  },
  "status": "SIGNED"
}
```

The bundle must be immutable after signing.

Any modification produces a different canonical representation and therefore a different cryptographic identity.

29.2 Policy Activation Transaction

Policy activation must be atomic.



There must never be a production interval in which different authorization workers unintentionally execute different policy versions for the same policy epoch.

29.3 Policy Epochs

Haven should maintain an explicit **policy epoch identifier**.

A policy epoch represents the exact policy state under which authorization decisions were expected to execute.

```
{
  "policy_epoch": "PE-2026-09-01-0007",
  "policy_bundle": "HAVEN-POLICY-BUNDLE-NG-2026-09",
  "policy_version": "2026.09.01",
  "policy_hash": "sha256:<hash>",
  "activated_at": "2026-09-01T00:00:00Z",
  "activation_status": "ACTIVE"
}
```

Every authorization event must capture the policy epoch.

This enables forensic reconstruction even when policy versions are later superseded.

29.4 Runtime Convergence

A distributed Haven deployment must continuously verify policy convergence.

Each authorization sidecar should expose a local policy-state record containing:

```
runtime_id
deployment_id
policy_epoch
policy_hash
compiler_version
loaded_at
activation_status
integrity_status
last_verified_at
```

The control plane compares these values across authorized execution nodes.

If a node diverges from the expected policy epoch, the node must enter a controlled governance state rather than silently continue executing unknown policy.

29.5 Policy Drift Detection

Policy drift occurs when the runtime policy state differs from the authorized control-plane state.

Examples include:

- unexpected policy hash;
- obsolete policy version;
- unsigned policy;
- expired policy;
- missing policy;
- incompatible compiler output;
- unauthorized local modification;
- runtime compilation mismatch.

The response must be deterministic:

```
EXPECTED HASH ≠ RUNTIME HASH
      ↓
POLICY INTEGRITY EVENT
      ↓
BLOCK NEW UNTRUSTED POLICY
      ↓
RETAIN LAST VALID POLICY IF AVAILABLE
      ↓
ESCALATE GOVERNANCE EVENT
      ↓
ENTER CONTROLLED STATE
```

The system must not automatically replace a valid production policy with an unverified policy simply because the control plane reports a newer version.

30. POLICY ROLLBACK & SAFE DEPLOYMENT

Policy deployment is security-sensitive infrastructure.

A defective application release can potentially be rolled back through conventional deployment tooling. A defective authorization policy can alter who is permitted to access data, execute workflows or cross organizational boundaries.

Haven therefore requires policy rollback to be treated as a governed security transaction.

30.1 Rollback Conditions

Rollback may be initiated when:

- a policy causes unexpected authorization failures;
- regression defects are discovered;
- a policy introduces unintended access;
- a regulatory mapping is withdrawn or corrected;
- a signing authority is compromised;
- policy/runtime incompatibility is detected;
- production telemetry identifies material policy drift.

30.2 Rollback Execution

```
ROLLBACK REQUEST
  ↓
AUTHORIZED OPERATOR
  ↓
REASON + INCIDENT REFERENCE
  ↓
TARGET POLICY VALIDATION
  ↓
SIGNATURE / HASH VERIFICATION
  ↓
REGRESSION TEST
  ↓
ATOMIC POLICY EPOCH CHANGE
  ↓
RUNTIME CONVERGENCE CHECK
  ↓
AUDIT EVENT
```

Rollback must not erase the existence of the superseded policy.

The audit system must retain:

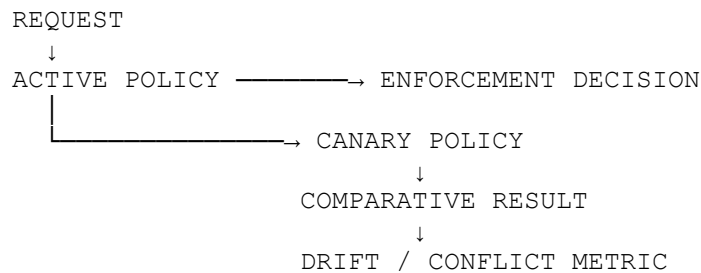
- original policy version;
- replacement policy version;
- reason for rollback;
- initiating authority;
- approval chain;
- activation timestamp;
- affected policy epoch;
- affected environments;
- verification result.

This preserves historical truth.

30.3 Canary Policy Activation

For high-risk policy changes, Haven should support controlled canary deployment.

A new policy may initially be evaluated in shadow mode:



The canary policy must not silently override the active policy.

Instead, it produces comparative telemetry such as:

```
{
  "request_id": "REQ-01J...",
  "active_policy_result": "ALLOW",
  "canary_policy_result": "DENY",
  "conflict": true,
  "materiality": "HIGH",
  "action": "NO_CHANGE",
  "review_required": true
}
```

This enables policy changes to be tested against real production patterns without turning the production population into an uncontrolled experiment.

31. ATTRIBUTE PROVENANCE, FRESHNESS & TRUST SCORING

ABAC decisions are only meaningful when the attributes used to produce them can be trusted.

Haven must therefore distinguish between **attribute existence** and **attribute authority**.

An attribute such as:

```
role = "CLINICIAN"
```

is insufficient by itself.

The authorization engine must understand:

- who asserted the attribute;
- when it was issued;
- whether the source is trusted;
- whether it has been revoked;
- whether it is still within its freshness window;
- whether it was cached;
- whether its integrity can be verified;
- what policy scopes the attribute is permitted to influence.

31.1 Attribute Provenance Object

```
{
  "attribute": "clinical_role",
  "value": "CLINICIAN",
  "subject_reference": "sub_01J...",
  "source": "AUTHORIZED_IDENTITY_PROVIDER",
  "assurance": "HIGH",
  "issued_at": "2026-09-03T08:00:00Z",
  "fresh_until": "2026-09-03T12:00:00Z",
  "revocation_status": "NOT_REVOKED",
  "integrity_status": "VERIFIED",
  "cache_status": "FRESH",
  "source_signature": "verified",
  "policy_scope": [
    "CLINICAL_ACCESS"
  ]
}
```

31.2 Attribute Trust Dimensions

Attribute trust must be evaluated across independent dimensions.

```
TRUST =  
  SOURCE AUTHORITY  
  × INTEGRITY  
  × FRESHNESS  
  × REVOCATION STATE  
  × SCOPE VALIDITY  
  × CONTEXTUAL RELEVANCE
```

This is a conceptual governance model rather than a mandatory mathematical scoring algorithm.

A production implementation may use deterministic categorical states instead of numerical scores.

Recommended states:

- VERIFIED
- TRUSTED
- CACHED_VALID
- STALE
- REVOKED
- CONFLICTING
- UNKNOWN

A REVOKED or CONFLICTING attribute must never be treated as equivalent to VERIFIED.

31.3 Attribute Conflict Resolution

Multiple authoritative systems may provide contradictory information.

Example:

```
Identity Provider A:  
clinical_role = CLINICIAN
```

```
Organization Registry:  
clinical_role = ADMINISTRATOR
```

```
Credential Registry:  
clinical_role = REVOKED
```

Haven must not resolve this conflict through arbitrary application logic.

The policy engine should instead evaluate provenance and authority precedence.

```
ATTRIBUTE CONFLICT
↓
PROVENANCE COMPARISON
↓
SOURCE AUTHORITY
↓
ISSUANCE / FRESHNESS
↓
REVOCATION CHECK
↓
POLICY PRECEDENCE
↓
RESOLVE / DENY / STEP-UP / REVIEW
```

Where a conflict cannot be deterministically resolved, the correct output is **INDETERMINATE**, not an optimistic authorization.

31.4 Attribute Revocation

Revocation is distinct from expiration.

An attribute may remain within its normal TTL while becoming invalid because its source has revoked it.

Therefore:

VALIDITY $\neq$ FRESHNESS

The degraded cache mechanism must never convert an expired or revoked attribute into a valid authorization merely because the cached record remains cryptographically intact.

32. ZERO-TRUST WORKFLOW STATE GOVERNANCE

Healthcare workflows are not single API calls.

A clinical coordination process may involve multiple organizations, services, devices and asynchronous transitions.

Haven must therefore authorize not only individual actions, but the **state transitions** that define a workflow.

32.1 Workflow State Authorization

A state transition should be represented as:

```
{
  "workflow_id": "WF-01J...",
  "current_state": "REFERRAL_ACCEPTED",
  "requested_state": "CARE_COORDINATION_ACTIVE",
  "initiating_principal": "sub_01J...",
  "service_identity": "svc_coordination_01",
  "purpose": "CARE_COORDINATION",
  "resource_classification": "CLINICAL_CRITICAL",
  "jurisdiction": "NG",
  "policy_epoch": "PE-2026-09-01-0007"
}
```

The PDP evaluates whether the transition itself is authorized.

This prevents a service from obtaining legitimate access to a resource but using that access to force the workflow into an unauthorized state.

32.2 State Transition Controls

Every critical workflow should define:

```
CURRENT STATE
  ↓
AUTHORIZED TRANSITION?
  ↓
REQUIRED ATTRIBUTES PRESENT?
  ↓
PURPOSE VALID?
  ↓
JURISDICTION VALID?
  ↓
RISK / SAFETY CONDITIONS
  ↓
NORMAL / DEGRADED / BREAK-GLASS
  ↓
NEXT STATE
```

Invalid transitions must produce a deterministic denial or controlled escalation.

Example:

```
REFERRAL_CREATED
  ↓
REFERRAL_ACCEPTED
  ↓
CARE_COORDINATION_ACTIVE
  ↓
CARE_EVENT_EXECUTED
  ↓
CARE_CLOSED
```

A system must not permit:

```
REFERRAL_CREATED
  ↓
CARE_CLOSED
```

merely because a downstream application possesses database write access.

32.3 Workflow Checkpoints

Long-running workflows should establish authorization checkpoints at material state boundaries.

Recommended checkpoints include:

- initiation;
- cross-organization handoff;
- sensitive-data disclosure;
- financial authorization;
- clinical escalation;
- emergency activation;
- completion;
- cancellation;
- reopening after closure.

This ensures that authorization is continuously evaluated against changing context rather than being granted once and implicitly trusted for the lifetime of the workflow.

33. GOVERNANCE FAILURE CLASSIFICATION

Haven must distinguish between different failure classes because not every failure should produce the same operational response.

33.1 Failure Taxonomy

Failure	Meaning	Default Response
ATTRIBUTE_UNAVAILABLE	Required attribute cannot be resolved	Degraded evaluation / restricted decision
ATTRIBUTE_REVOKED	Attribute authority revoked	Deny affected authority
POLICY_INVALID	Policy fails structural validation	Reject policy
POLICY_SIGNATURE_INVALID	Policy signer cannot be trusted	Reject policy
POLICY_EXPIRED	Policy validity period ended	Review / controlled state
POLICY_CONFLICT	Rules produce unresolved conflict	Deny or review
PDP_UNAVAILABLE	Local decision engine unavailable	Activate resilience pathway
PEP_FAILURE	Enforcement boundary failed	Block unverified execution
AUDIT_QUEUE_FAILURE	Evidence queue unavailable	Buffer/retry; do not block solely for ledger finality
LEDGER_ANCHOR_FAILURE	Cryptographic anchor unavailable	Retain unanchored evidence and retry
BREAK_GLASS_EXPIRED	Emergency authority expired	Terminate emergency scope
JURISDICTION_UNKNOWN	Applicable jurisdiction cannot be established	Restrict / review

33.2 Failure-State Principle

The system must answer two separate questions:

Can the requested action be safely authorized?

and

Can the authorization event currently be recorded and anchored?

These questions must never be coupled unnecessarily.

A temporary ledger-anchor outage must not automatically become a clinical authorization outage.

Conversely, an authorization-policy integrity failure must not be concealed merely because the audit pipeline remains operational.

33.3 Governance Incident Envelope

```
{
  "incident_id": "GOV-01J...",
  "failure_class": "POLICY_SIGNATURE_INVALID",
  "severity": "CRITICAL",
  "detected_at": "2026-09-03T08:45:00Z",
  "runtime_id": "pdp-prod-07",
  "policy_id": "HAVEN-NG-PRIVACY",
  "policy_version": "2026.09.01",
  "expected_hash": "sha256:<expected>",
  "observed_hash": "sha256:<observed>",
  "action": "POLICY_REJECTED",
  "fallback_state": "NORMAL_USING_LAST_VALID_POLICY",
  "audit_event_required": true
}
```

Every governance failure must itself become auditable infrastructure.

34. FORENSIC RECONSTRUCTION & COMPLIANCE EVIDENCE GRAPH

The Haven governance architecture must allow an authorized investigator, auditor, security team or compliance function to reconstruct a consequential system action without relying on application logs alone.

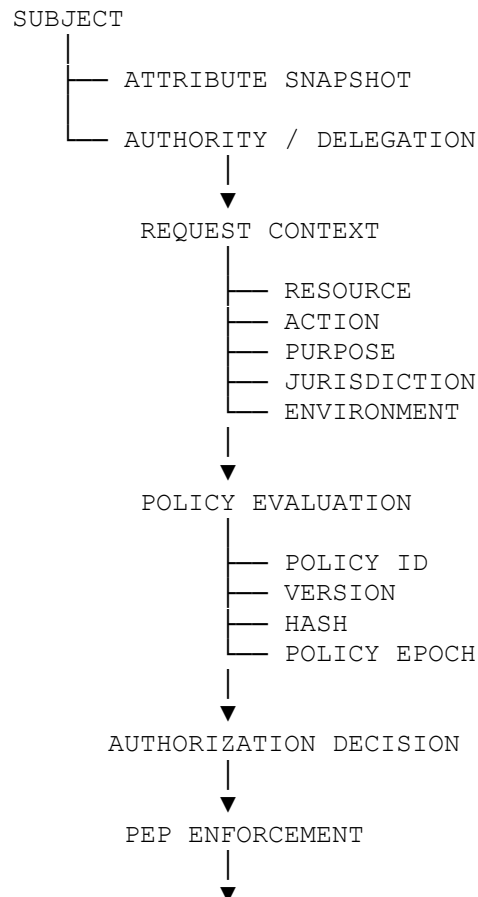
The objective is not merely to prove that an event occurred.

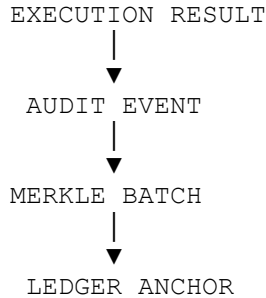
The objective is to establish:

who acted, what was requested, which authority existed, which policy was active, what attributes were evaluated, what decision was issued, what was actually executed, and what cryptographic evidence subsequently established the integrity of the record.

34.1 Evidence Graph

Every consequential authorization should produce a linked evidence graph:





Each node should possess a stable identifier enabling deterministic correlation.

34.2 Evidence Correlation Object

```
{
  "evidence_id": "EVD-01J...",
  "request_id": "REQ-01J...",
  "authorization_id": "AUTH-01J...",
  "subject_reference": "sub_01J...",
  "resource_reference": "res_01J...",
  "workflow_id": "WF-01J...",
  "policy_epoch": "PE-2026-09-01-0007",
  "policy_hash": "sha256:<policy-hash>",
  "decision": "ALLOW_WITH_CONSTRAINTS",
  "pep_result": "ENFORCED",
  "execution_result": "SUCCESS",
  "audit_event_id": "EVT-01J...",
  "merkle_batch_id": "MB-01J...",
  "ledger_anchor_id": "ANC-01J..."
}
```

The evidence graph must not require replication of the underlying clinical database.

Instead, it should contain sufficient references and cryptographic proofs to establish the integrity of the decision without unnecessarily duplicating sensitive payloads.

34.3 Minimal Evidence Principle

Haven should collect the **minimum evidence necessary to prove the authorization decision**.

Audit events should preferentially reference:

- pseudonymous subject identifiers;
- resource references;
- attribute identifiers rather than unnecessary attribute values;
- policy identifiers;
- policy hashes;
- decision metadata;
- workflow identifiers;

- timestamps;
- jurisdiction;
- operational state;
- cryptographic hashes.

Sensitive clinical content should not be copied into the audit ledger merely because an authorization event occurred.

This maintains the distinction between:

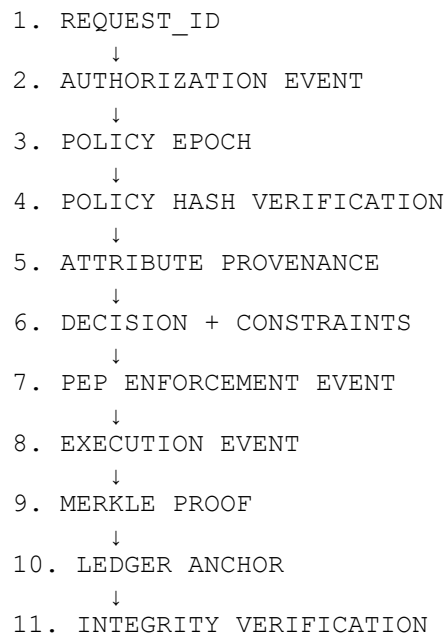
evidence of access

and

the underlying clinical data accessed.

34.4 Forensic Reconstruction Procedure

An authorized investigation should follow a deterministic process:



If any link is missing, the evidence state must identify the missing component rather than silently treating the chain as complete.

34.5 Evidence Integrity States

COMPLETE
INCOMPLETE
PENDING_ANCHOR
ANCHOR_FAILED
INTEGRITY_FAILURE
QUARANTINED
UNDER_REVIEW
VERIFIED

VERIFIED should only be assigned after cryptographic verification has successfully established the expected relationship between the event, Merkle proof and anchor.

35. AUDIT STREAM PARTITIONING & MULTI-TENANT ISOLATION

Haven may coordinate events across multiple institutions while maintaining organizational separation.

The audit architecture must therefore prevent one tenant's governance stream from becoming an implicit access path into another tenant's information.

35.1 Stream Identity

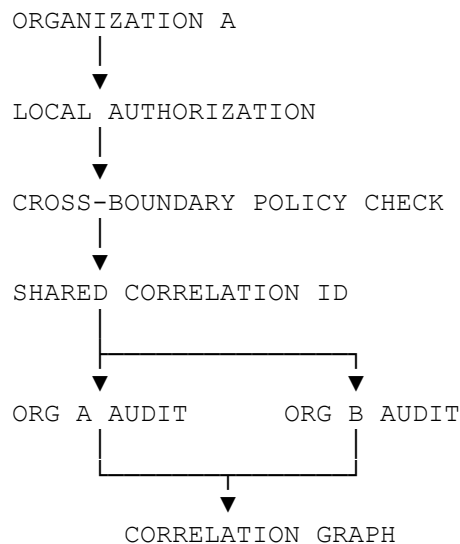
Each audit stream should be logically partitioned by governance boundary.

```
{
  "stream_id": "STREAM-ORG-001",
  "organization_id": "ORG-001",
  "jurisdiction": "NG",
  "environment": "PRODUCTION",
  "classification": "REGULATED",
  "stream_epoch": "2026-09"
}
```

A stream may contain references to cross-organizational events where required for correlation, but the authorization model must continue to enforce tenant boundaries.

35.2 Cross-Organization Evidence

For a transaction involving Organization A and Organization B:



The correlation identifier permits participating parties to establish that events belong to the same workflow without automatically exposing each party's complete internal audit stream.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

35.3 Tenant Isolation Requirements

The implementation must enforce:

- tenant-scoped authorization;
- tenant-scoped encryption;
- tenant-scoped access policies;
- tenant-aware audit filtering;
- separation of administrative authority;
- explicit cross-tenant delegation;
- independent evidence access controls.

A platform administrator should not automatically inherit unrestricted access to all regulated tenant evidence merely by possessing infrastructure-level privileges.

35.4 Evidence Disclosure

Audit evidence itself is regulated information where it contains personal or sensitive references.

Therefore, access to audit evidence must pass through the same governance architecture.

The system must distinguish:

```
INFRASTRUCTURE ADMINISTRATION
    ≠
CLINICAL DATA ACCESS
    ≠
AUDIT EVIDENCE ACCESS
    ≠
REGULATORY INVESTIGATION AUTHORITY
```

Each requires separately governed authority.

36. COMPLIANCE EVIDENCE RETENTION & INTEGRITY

Haven must define retention behavior for governance evidence independently from application-level clinical retention policies.

A compliance event may need to remain verifiable even after the underlying application record has been modified, archived or deleted in accordance with an applicable lifecycle policy.

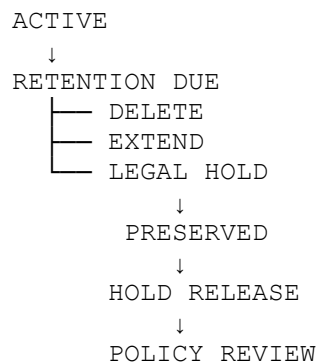
36.1 Evidence Retention Object

```
{
  "evidence_class": "AUTHORIZATION_DECISION",
  "retention_policy_id": "RET-AUTH-001",
  "created_at": "2026-09-03T08:42:15Z",
  "retention_state": "ACTIVE",
  "expiration_at": "policy-derived",
  "legal_hold": false,
  "anchor_status": "ANCHORED",
  "deletion_authority": "POLICY_CONTROLLED"
}
```

Retention duration must be configurable through policy rather than permanently embedded in application code.

36.2 Legal Hold

Where an applicable governance process requires preservation of evidence, a legal-hold state may prevent ordinary lifecycle deletion.



A legal hold must itself become an auditable governance event.

36.3 Evidence Destruction

Evidence deletion, where permitted, must never be implemented as an unlogged storage operation.

The system must generate an event describing:

- evidence identifier;
- policy basis;
- authorization authority;
- retention rule;
- deletion timestamp;
- affected storage scope;
- resulting evidence state.

Cryptographic anchors may continue to establish that a prior event existed even when the underlying detailed evidence has subsequently reached an authorized retention endpoint.

37. GOVERNANCE ACCESS TO GOVERNANCE

The final principle of zero-trust governance is recursive:

The governance system itself must be governed.

No administrator, security operator, policy engineer or infrastructure service should possess implicit authority to modify governance state without an explicit authorization context.

Administrative operations must therefore be treated as high-value protected resources.

Examples include:

- policy activation;
- policy rollback;
- signer-key changes;
- ABAC configuration changes;
- emergency-authority configuration;
- attribute-source registration;
- jurisdiction-policy activation;
- audit-stream configuration;
- retention-rule changes;
- ledger-anchor configuration.

Every such operation requires:

IDENTITY
+
ADMINISTRATIVE ROLE
+
AUTHORITY SCOPE
+
PURPOSE
+
STEP-UP ASSURANCE
+
CHANGE REFERENCE
+
POLICY VALIDATION
+
AUDIT EVIDENCE

This prevents the control plane from becoming a privileged exception to the very governance model it imposes on the rest of the system.

38. SEPARATION OF DUTIES & GOVERNANCE AUTHORITY MODEL

Haven should implement administrative separation of duties for security-critical governance operations.

No single administrative identity should be able to silently originate, approve, activate and erase evidence of a high-impact policy change.

38.1 Governance Roles

A reference authority model includes:

Role	Primary Authority	Restricted Authority
Policy Author	Draft policy	Cannot independently activate
Policy Reviewer	Review technical/legal mapping	Cannot authoritatively activate
Security Approver	Security validation	Cannot alter approved policy content
Release Operator	Production activation	Cannot modify policy semantics
Incident Commander	Emergency governance	Cannot erase emergency evidence
Auditor	Evidence inspection	Cannot alter execution state
Key Custodian	Signing authority	Cannot independently author policy
Platform Administrator	Infrastructure operations	Cannot bypass PEP

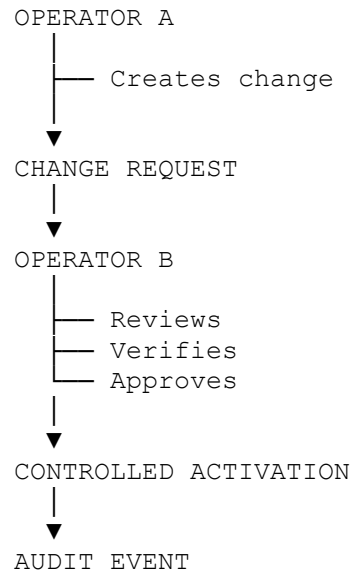
These roles may be adapted to organizational requirements, but the principle remains constant: **critical governance authority must be compartmentalized.**

38.2 Dual-Control Operations

The following operations should support dual-control authorization where risk warrants it:

- production policy activation;
- signing-key rotation;
- emergency-policy modification;
- break-glass authority modification;
- audit retention changes;
- ledger-anchor configuration;
- cross-jurisdiction policy activation;
- administrative bypass configuration.

Example:



The approval chain must be cryptographically associated with the resulting governance event.

39. BREAK-GLASS AUTHORITY GOVERNANCE

Emergency authorization is one of the highest-risk capabilities in the Haven architecture.

Accordingly, the system must distinguish between:

Emergency Authority Configuration

and

Emergency Authority Invocation.

Changing who may invoke emergency authority is itself a privileged governance action.

39.1 Break-Glass Authority Object

```
{
  "authority_id": "BGA-001",
  "organization_id": "ORG-001",
  "authorized_roles": [
    "EMERGENCY_CLINICAL_AUTHORITY"
  ],
  "permitted_purposes": [
    "EMERGENCY_TREATMENT"
  ],
  "maximum_duration_seconds": 1800,
  "maximum_scope": [
    "CLINICAL_CRITICAL"
  ],
  "step_up_required": true,
  "post_event_review_required": true,
  "status": "ACTIVE"
}
```

The runtime should never accept an emergency request merely because the requester has an ordinary administrative role.

39.2 Emergency Scope Intersection

Emergency authority is still subject to multiple constraints:

EMERGENCY AUTHORITY
 $\cap$
SUBJECT AUTHORITY
 $\cap$
RESOURCE CLASSIFICATION
 $\cap$
PERMITTED PURPOSE
 $\cap$
JURISDICTIONAL POLICY

$\cap$
TIME WINDOW
 $\cap$
ORGANIZATIONAL POLICY
 $\cap$
CLINICAL CRITICALITY

The resulting scope is the maximum authority available during the emergency.

Break-glass must never become a universal administrative bypass.

39.3 Emergency Termination

Emergency authority must automatically expire.

BREAK-GLASS ACTIVATED
↓
ACTIVE WINDOW
↓
WARNING THRESHOLD
↓
EXPIRATION
↓
AUTHORITY REVOKED
↓
POST-EVENT REVIEW

Any request received after expiration must produce:

```
{
  "error": {
    "code": "BREAK_GLASS_EXPIRED",
    "message": "Emergency authorization window has expired.",
    "retryable": false,
    "requires_reauthorization": true
  }
}
```

39.4 Emergency Abuse Detection

The governance telemetry layer should identify abnormal patterns including:

- repeated break-glass activation by one identity;
- unusually long emergency sessions;
- repeated activation for the same resource;
- emergency authority outside expected workflow patterns;
- emergency access during non-emergency operational states;
- repeated access to unrelated organizations;
- emergency activation immediately following denied requests.

These signals should generate risk events rather than automatically terminating legitimate clinical activity.

40. POLICY SIMULATION & DIGITAL GOVERNANCE TEST ENVIRONMENT

Before production activation, policy changes should be executable against representative historical or synthetic authorization events.

The objective is to determine whether a policy changes system behavior before that policy becomes authoritative.

40.1 Simulation Pipeline

```
PROPOSED_POLICY
↓
COMPILE
↓
STATIC_VALIDATION
↓
REGRESSION_SUITE
↓
REPRESENTATIVE_EVENT_CORPUS
↓
SIMULATION
↓
COMPARE_WITH_ACTIVE_POLICY
↓
CLASSIFY_DIFFERENCES
↓
APPROVAL / REJECTION
```

40.2 Simulation Difference Classes

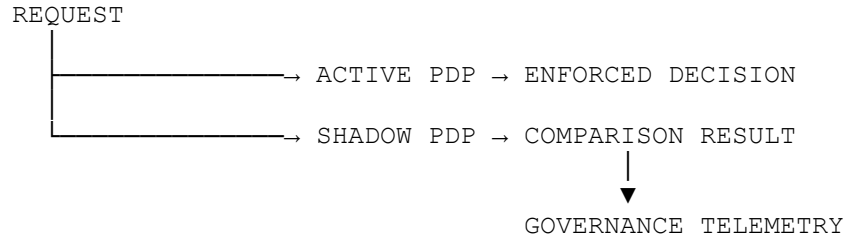
Every difference should be classified:

```
EXPECTED_CHANGE
BENIGN_CHANGE
RESTRICTIVE_CHANGE
PERMISSION_EXPANSION
UNEXPECTED_DENIAL
UNEXPECTED_ACCESS
JURISDICTIONAL_CONFLICT
SAFETY_IMPACT
UNRESOLVED
```

Permission expansions should receive greater scrutiny than equivalent restrictions because an authorization policy that unintentionally expands access can create a materially different security posture.

40.3 Shadow Evaluation

The production PDP may optionally evaluate an incoming request against a proposed policy without allowing that proposed policy to control execution.



Shadow evaluation enables Haven to observe the practical effect of regulatory or organizational policy changes before activation.

41. ARCHITECTURAL SLO & GOVERNANCE ASSURANCE TARGETS

The governance architecture should be measured as production infrastructure rather than treated solely as a compliance artifact.

41.1 Core Targets

Metric	Target
Authorization overhead	<10 ms target
Local PDP availability	≥99.99% target
Policy activation integrity	100% signed/verified
Unauthorized policy activation	0
Audit event loss	0 accepted
Audit queue durability	Durable before acknowledgment where configured
Merkle batch interval	~5 minutes default
Break-glass availability	High-availability target
Policy convergence	Defined deployment-specific SLO
Unresolved policy conflicts	0 permitted for activation
Cryptographic verification failures	0 accepted for trusted evidence

These are architectural targets rather than guarantees. Production service-level commitments must be established through deployment-specific capacity, testing and operational agreements.

41.2 Governance Health Scorecard

The operational dashboard should expose:

AUTHORIZATION

—	P50 / P95 / P99 LATENCY
—	DECISION RATE
—	DENIAL RATE
—	PDP ERRORS

POLICY

—	ACTIVE EPOCH
—	HASH CONSISTENCY
—	EXPIRATION STATUS
—	DRIFT
—	ROLLBACKS

ATTRIBUTES

- FRESH
- CACHED
- STALE
- REVOKED
- CONFLICTING

EMERGENCY

- ACTIVE
- EXPIRING
- EXPIRED
- UNDER REVIEW

AUDIT

- QUEUE DEPTH
- OLDEST UNPROCESSED EVENT
- MERKLE BATCHES
- ANCHOR STATUS
- EVIDENCE INTEGRITY

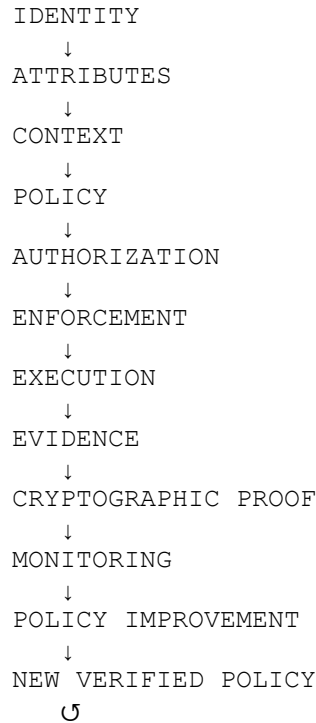
SECURITY

- POLICY INTEGRITY EVENTS
- ADMINISTRATIVE CHANGES
- CROSS-BOUNDARY EVENTS
- HIGH-RISK AUTHORIZATION EVENTS

The dashboard itself must not become a privileged bypass surface. Every administrative action originating from the governance interface must continue through the same authorization architecture.

42. GOVERNANCE ARCHITECTURE PRINCIPLE

Haven's zero-trust model ultimately establishes a closed governance loop:



The system therefore does not treat compliance as a report generated after an event.

It makes governance part of the execution substrate.

The architectural objective is a healthcare coordination environment in which authorization is explicit, regulatory constraints are configurable and versioned, emergency access is controlled, policy changes are governed, audit evidence is independently verifiable, and operational resilience does not require abandoning security principles.

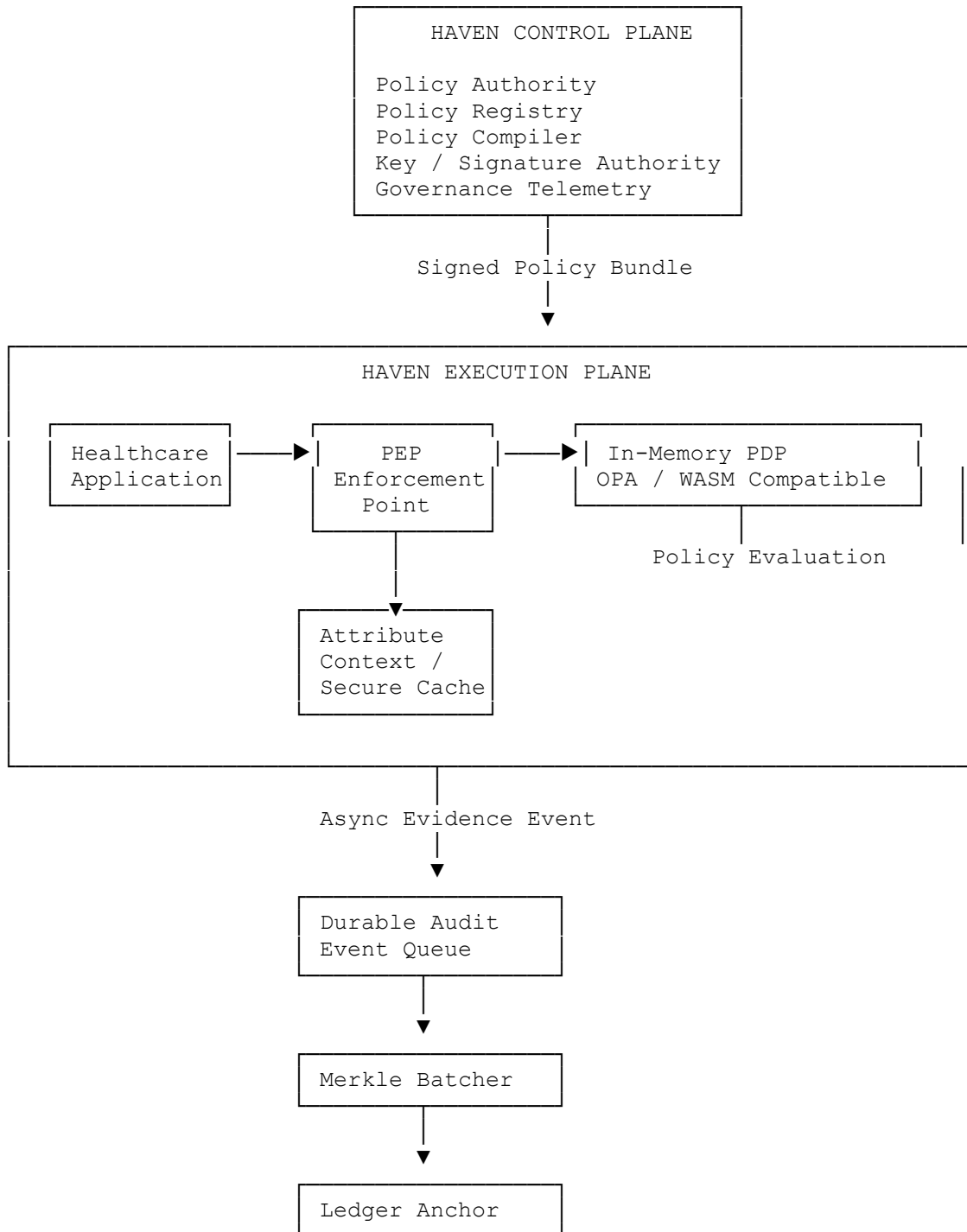
The resulting architecture provides Haven with a durable governance foundation capable of operating across fragmented healthcare environments while preserving a consistent trust model across organizations, jurisdictions, services and clinical workflows.

43. PRODUCTION IMPLEMENTATION REFERENCE ARCHITECTURE

The Haven Zero-Trust Governance Framework is intended to operate as an infrastructure control layer beneath participating healthcare applications.

The implementation must preserve a strict separation between application functionality and governance authority.

43.1 Production Component Topology



The architecture is deliberately divided into:

Control Plane → Execution Plane → Evidence Plane.

The control plane determines which policies are trusted.

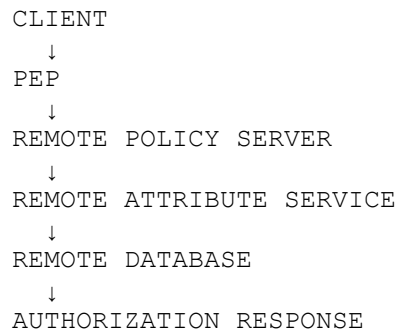
The execution plane determines whether individual actions satisfy those policies.

The evidence plane establishes durable, independently verifiable evidence of what occurred.

43.2 Deployment Rule

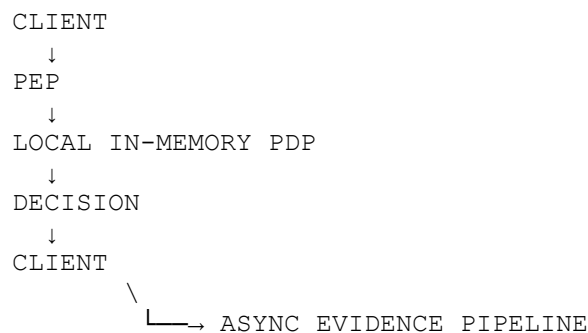
A production deployment must not place a remote control-plane dependency directly inside the authorization request path.

The following architecture is prohibited:



This creates unnecessary latency and introduces additional availability dependencies.

The preferred model is:



Remote systems may participate in policy distribution, attribute synchronization and governance operations, but they should not be required for every authorization decision.

44. IMPLEMENTATION CONTRACTS

Every production Haven deployment should implement the following mandatory contracts.

44.1 Authorization Contract

INPUT

- Subject
- Resource
- Action
- Purpose
- Environment
- Jurisdiction
- Workflow State
- Operational State

PROCESS

- Normalize
- Validate
- Resolve Policy
- Evaluate ABAC
- Apply Constraints
- Produce Decision

OUTPUT

- Decision
- Policy Version
- Policy Hash
- Constraints
- Decision ID
- Audit Reference

44.2 Evidence Contract

INPUT

- Authorization Decision
- Policy Context
- Attribute References
- Enforcement Result
- Execution Result

PROCESS

- Canonicalize
- Hash
- Queue
- Batch
- Anchor

OUTPUT

- Event ID
- Event Hash
- Batch ID
- Merkle Root
- Anchor Reference

44.3 Emergency Contract

INPUT

- Emergency Declaration
- Requester
- Authority
- Reason
- Resource
- Purpose
- Requested Scope

PROCESS

- Validate Authority
- Validate Criticality
- Validate Scope
- Validate Duration
- Issue Temporary Authority
- Generate Evidence

OUTPUT

- Break-Glass ID
- Decision
- Expiration
- Scope
- Audit Reference
- Review Requirement

45. ENGINEERING ACCEPTANCE CRITERIA

The framework should not be considered production-ready until the following conditions have been demonstrated.

Authorization

- ABAC decisions execute through the local in-memory policy runtime.
- Authorization overhead remains within the defined **<10ms target** under the deployment's tested workload.
- No mandatory synchronous PIP network call exists in the critical authorization path.
- Every consequential request passes through an enforceable PEP.
- Authorization decisions contain a policy version or epoch reference.
- Constraint-based decisions are enforceable rather than advisory.

Resilience

- NORMAL state operates against current verified attributes.
- DEGRADED state uses encrypted, integrity-verified cached attributes.
- Attribute TTLs are policy-controlled.
- Revoked attributes cannot be treated as valid cached authority.
- EMERGENCY BREAK-GLASS provides bounded, temporary authority.
- Emergency authority automatically expires.
- Critical clinical workflows have an auditable emergency pathway.

Policy Governance

- Every production policy is cryptographically signed.
- Policy hashes are verified before activation.
- Policy activation is atomic.
- Runtime policy convergence is observable.
- Rollback is versioned and auditable.
- Policy changes pass regression testing.
- Unauthorized policy modification is rejected.

Audit

- Authorization evidence is emitted asynchronously.
- The synchronous path does not wait for ledger finality.
- Audit events use canonical serialization.
- Event hashes are deterministic.
- Merkle batching is operationally observable.
- Ledger anchoring operates on a scheduled cadence, with **five minutes as the default reference interval**.
- Anchor failures do not silently destroy pending evidence.

- Evidence integrity can be independently verified.

Compliance

- Regulatory requirements are represented through versioned declarative policy packs.
- Jurisdiction selection is explicit.
- NDPA-derived controls are technically mapped without embedding statutory interpretation directly into binaries.
- Policy changes preserve historical versions.
- Compliance evidence can be correlated to authorization decisions.
- Data-subject workflows remain policy-controlled.
- Compliance administration cannot bypass the authorization layer.

46. SECURITY VALIDATION TEST MATRIX

Before activation, Haven should execute a deterministic security test suite.

Test	Expected Result
Valid identity + valid policy	ALLOW where authorized
Missing required attribute	DENY / DEFER according to policy
Revoked credential	DENY
Expired attribute	Restricted / DENY
Invalid policy signature	Policy rejected
Modified policy hash	Policy rejected
Unknown jurisdiction	Restricted / REVIEW_REQUIRED
Conflicting authoritative attributes	INDETERMINATE / REVIEW
Unauthorized cross-tenant request	DENY
Invalid workflow transition	DENY
Expired break-glass authority	DENY
Valid emergency request	Bounded BREAK-GLASS authorization
Audit queue unavailable	Durable retry / buffering path
Ledger unavailable	Evidence retained for later anchoring
PEP failure	No unverified execution
Policy rollback	New policy epoch + audit evidence
Runtime policy drift	Governance alert + controlled response
Replay of prior authorization	REPLAY_REJECTED
Excessive emergency usage	Risk event generated
Unauthorized administrative change	DENY + security event

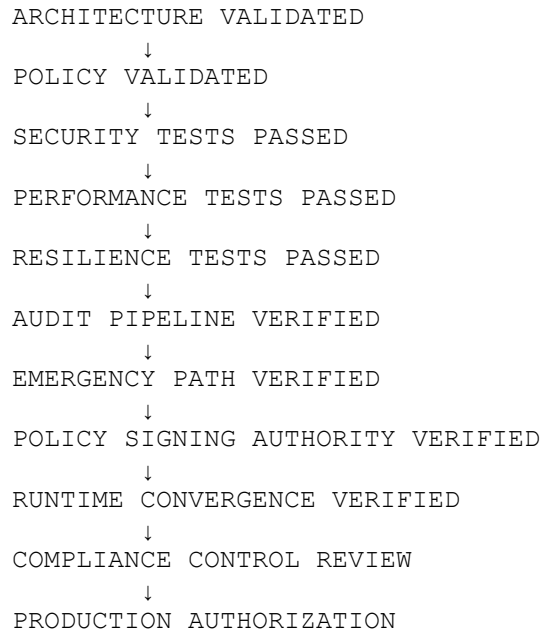
The test suite should execute during:

- initial implementation;
- policy activation;
- major runtime upgrades;
- security configuration changes;
- jurisdictional policy changes;
- emergency-control modifications.

The result of each production policy release should be retained as governance evidence.

47. PRODUCTION READINESS GATE

Haven should establish a formal **Governance Readiness Gate** before permitting a new environment to process regulated workflows.



No individual control should be marked complete solely because the corresponding software component exists.

The control must be **implemented, exercised, observed and evidenced**.

48. FINAL ZERO-TRUST GOVERNANCE DIRECTIVE

Haven Protocol's security and compliance architecture is founded on a single operational principle:

No consequential action is trusted merely because the requester, service, application, network or organization is trusted.

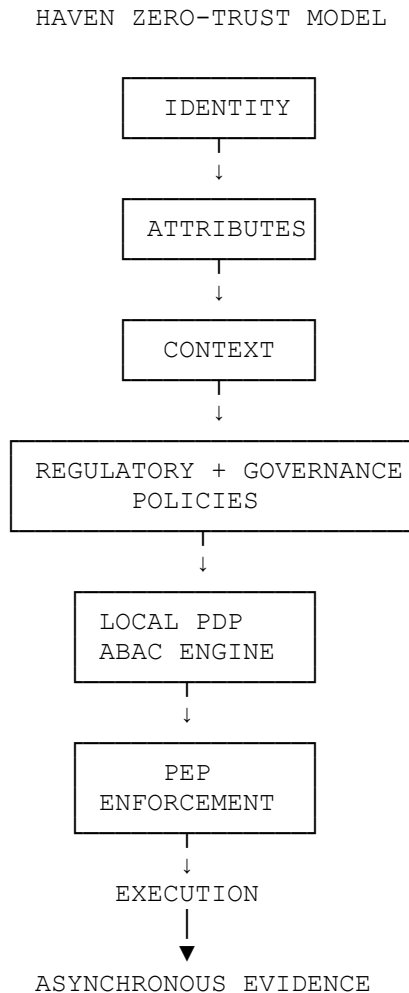
Authority must be evaluated from current context.

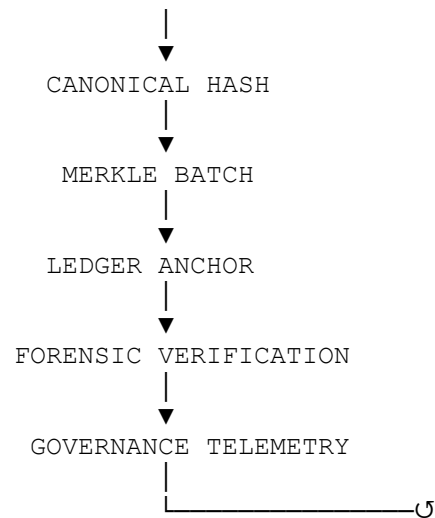
That context includes identity, attributes, resource classification, requested action, purpose, jurisdiction, workflow state, regulatory policy, risk conditions and operational state.

The resulting decision must be enforceable at the boundary where the action occurs.

The evidence of that decision must then be independently preserved.

48.1 Complete Governance Model





49. THE HAVEN COMPLIANCE MOAT

The long-term defensibility of Haven does not depend on claiming ownership over healthcare data.

It depends on making governance infrastructure increasingly difficult to reproduce correctly at production scale.

The defensible architecture consists of interconnected layers:

Layer 01 — Attribute Taxonomy

A normalized representation of subjects, services, resources, purposes, jurisdictions, workflow states and operational conditions.

Layer 02 — Policy Execution

A deterministic ABAC runtime capable of evaluating those attributes at the enforcement boundary with production-grade latency.

Layer 03 — Regulatory Abstraction

Versioned jurisdictional policy packs capable of translating approved regulatory interpretations into machine-executable constraints.

Layer 04 — Clinical Resilience

A controlled NORMAL → DEGRADED → EMERGENCY BREAK-GLASS state engine that preserves clinical continuity without abandoning governance.

Layer 05 — Evidence Infrastructure

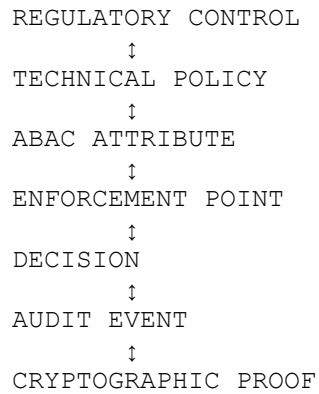
An asynchronous event architecture capable of converting authorization activity into cryptographically verifiable evidence.

Layer 06 — Policy Governance

Signed policies, controlled activation, policy epochs, rollback, runtime convergence and drift detection.

Layer 07 — Compliance Intelligence

A continuously maintained relationship between:



This relationship becomes progressively more valuable as Haven operates across institutions, jurisdictions and increasingly complex healthcare workflows.

The moat is therefore **governance depth, interoperability, evidence continuity and operational trust**.

It must never depend on unlawful exclusion, artificial obstruction, suppression of interoperability or preventing customers from exercising legitimate rights.

50. FINAL ARCHITECTURAL PRINCIPLES

The Haven Zero-Trust Technical Governance & ABAC Compliance Framework establishes the following non-negotiable principles:

01 — NEVER TRUST BY DEFAULT

No identity, service, application, network or organization receives implicit authority.

02 — AUTHORIZE AT THE ENFORCEMENT BOUNDARY

A policy that cannot be technically enforced is not an effective runtime control.

03 — KEEP THE PDP LOCAL

Authorization must execute through an in-memory policy runtime without mandatory synchronous external PIP calls.

04 — PRESERVE THE <10MS TARGET

Governance must be engineered as production infrastructure rather than an external compliance bottleneck.

05 — SEPARATE LAW FROM CODE

Regulatory interpretation belongs in reviewed, versioned declarative policy packs rather than hardcoded statutory logic inside application binaries.

06 — PRESERVE CLINICAL CONTINUITY

Critical workflows must have a controlled emergency pathway rather than being trapped indefinitely behind an unreviewed hard FAIL_CLOSED state.

07 — BREAK-GLASS IS GOVERNED AUTHORITY

Emergency access is temporary, scoped, observable, attributable and reviewable.

08 — AUDIT ASYNCHRONOUSLY

Clinical execution must not wait for Merkle construction, ledger consensus or anchor finality.

09 — MAKE EVIDENCE TAMPER-EVIDENT

Authorization history must be reconstructable through canonical events, cryptographic hashes, Merkle proofs and ledger anchors.

10 — VERSION EVERYTHING THAT MATTERS

Policies, attributes, governance states, emergency authorities and evidence schemas must possess identifiable versions or epochs.

11 — FAIL SAFELY

Invalid policies, compromised signatures, revoked attributes and enforcement failures must never silently produce expanded authority.

12 — GOVERN THE GOVERNANCE LAYER

Policy administrators, security operators and infrastructure administrators remain subject to zero-trust authorization.

13 — MINIMIZE DATA

Compliance evidence should establish that an action occurred without unnecessarily reproducing the underlying clinical payload.

14 — MAKE EVERY DECISION FORENSICALLY RECONSTRUCTABLE

A consequential decision should be explainable through its request context, policy, attributes, enforcement result and cryptographic evidence.

15 — TREAT COMPLIANCE AS EXECUTABLE INFRASTRUCTURE

Compliance is not a PDF generated after the fact.

It is a continuously evaluated control system embedded within the execution architecture.

DOCUMENT 4 — CLOSING STATEMENT

Haven Protocol's governance layer is designed to transform fragmented healthcare environments into a coordinated trust domain without requiring a single centralized owner of every underlying system or data store.

Its architectural authority comes from the enforcement boundary.

Its resilience comes from controlled operational states.

Its compliance posture comes from policy abstraction.

Its accountability comes from asynchronous immutable evidence.

Its defensibility comes from the accumulation of interoperable governance infrastructure.

The result is a protocol architecture in which **authorization, privacy, clinical safety, regulatory policy, operational resilience and forensic accountability are not separate products.**

They are coordinated execution properties of the same infrastructure layer.

HAVEN PROTOCOL

ZERO-TRUST TECHNICAL GOVERNANCE & ABAC COMPLIANCE FRAMEWORK

DOCUMENT 04 OF THE HAVEN TECHNICAL GOVERNANCE SERIES

**STATUS: ARCHITECTURAL REFERENCE / PRODUCTION GOVERNANCE
BASELINE**

END OF DOCUMENT 4